



**Politechnika
Śląska**

PROJEKT INŻYNIERSKI

Wybrane elementy kanału PDSCH systemu 5G NR modelowane w języku
opisu sprzętu

Krzysztof WOJTASIK

Nr albumu: 300964

Kierunek: Teleinformatyka

PROWADZĄCY PRACĘ

dr hab. inż., prof. PŚ Wojciech Sułek

KATEDRA Telekomunikacji i Teleinformatyki

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2026

Tytuł pracy

Wybrane elementy kanału PDSCH systemu 5G NR modelowane w języku opisu sprzętu

Streszczenie

Praca dotyczy modelowania wybranych elementów toru nadawczego kanału PDSCH (Physical Downlink Shared Channel) w systemie 5G NR w oparciu o założenia specyfikacji 3GPP. Celem projektu było opracowanie sprzętowych modeli bloków przetwarzania danych stanowiących kluczowe etapy przygotowania strumienia bitów do transmisji w warstwie fizycznej. Zrealizowano moduły odpowiedzialne za wyznaczanie sumy kontrolnej CRC (Cyclic Redundancy Check) oraz kodowanie kanałowe LDPC (Low-Density Parity-Check) stosowane w 5G NR. Implementację wykonano w języku Verilog, z podziałem na niezależne moduły możliwe do integracji w jeden łańcuch przetwarzania po stronie nadajnika.

Dodatkowo zaimplementowano załączkową funkcjonalność związaną z etapem puncturingu, polegającą na eliminacji bitów typu filler oraz pomijaniu na wyjściu dwóch pierwszych bloków informacji zgodnie z przyjętymi założeniami przetwarzania po kodowaniu.

W ramach pracy przygotowano środowisko symulacyjne oraz zestawy wektorów testowych generowanych w środowisku MATLAB, umożliwiające porównanie wyników modułów Verilog z danymi referencyjnymi. Poprawność działania zweryfikowano funkcjonalnie w symulacji na wielu przypadkach testowych, uwzględniających różne długości danych oraz sytuacje brzegowe charakterystyczne dla przetwarzania blokowego.

Planowany w założeniach projektu blok scramblingu (scrambling) nie został finalnie zaimplementowany; jego rola została omówiona, a możliwość rozbudowy systemu o ten etap wskazana jako kierunek dalszych prac.

Słowa kluczowe

Kodowanie, 5G NR, Physical Downlink Shared Channel, Verilog, FPGA, Low Density Parity-Check Codes

Thesis title

Selected Elements of the 5G NR PDSCH Channel Modeled in a Hardware Description Language

Abstract

This thesis presents the modeling of selected transmitter-side processing blocks of the 5G NR (New Radio) physical channel PDSCH (Physical Downlink Shared Channel) based on the assumptions of the 3GPP specification. The main objective was to develop

hardware-oriented models of key stages used to prepare a bitstream for transmission in the physical layer. The implemented blocks include CRC (Cyclic Redundancy Check) generation and 5G NR channel coding using LDPC (Low-Density Parity-Check) codes. The solution was implemented in Verilog as a set of independent modules that can be integrated into a single processing chain for the transmitter.

In addition, an initial puncturing-related mechanism was implemented. It removes filler bits and suppresses the first two information blocks at the output, according to the adopted post-coding processing assumptions.

A simulation environment and test workflow were prepared, with reference test vectors generated in MATLAB, enabling direct comparison between Verilog simulation outputs and golden data. Functional correctness was verified in simulation for multiple test cases, including different payload lengths and boundary conditions typical for block-based processing.

The scrambling block planned in the initial project scope was not implemented in the final version. Its role is discussed, and extending the design with scrambling is indicated as future work.

Key words

Coding, 5G NR, Physical Downlink Shared Channel, Verilog, FPGA, Low Density Parity-Check Codes

Spis treści

1	Wstęp	1
2	Analiza tematu	3
3	Wymagania i narzędzia	5
3.1	Założenia projektowe	5
3.1.1	Sposób przetwarzania danych	5
3.1.2	Model pracy	5
3.1.3	Sterowanie przepływem danych	5
3.1.4	Sposób podawania danych wejściowych	6
3.1.5	Elastyczność kodowania LDPC i obsługa różnych TB	6
3.1.6	Wymagania	7
3.2	Interfejsy i format danych	7
3.2.1	Interfejs wejściowy	7
3.2.2	Interfejs wyjściowy	8
3.3	Środowisko projektowe i narzędzia	8
3.3.1	Quartus Prime Lite Edition	9
3.3.2	Questa Intel Starter FPGA Edition	9
3.3.3	MATLAB	9
4	Koncepcja i realizacja projektu	11
4.1	Założenia architektoniczne toru	11
4.1.1	Stała magistrala 64-bit jako oś projektu	11
4.1.2	Uproszczony model sterowania strumieniem	11
4.1.3	Przetwarzanie blokowe i reset jako granica transakcji	12
4.1.4	Elastyczność kodowania LDPC	12
4.1.5	Weryfikacja poprzez symulację i synteżowalność	12
4.2	Struktura funkcjonalna i przepływ danych	12
4.2.1	Ogólny podział toru na etapy	13
4.2.2	Przepływ danych wejściowych i moment rozpoczęcia przetwarzania	13
4.2.3	Przepływ danych wewnątrz toru	13

4.2.4	Strumień wyjściowy i zakończenie przetwarzania	14
4.2.5	Warianty długości TB i konsekwencje dla przepływu danych	14
4.3	Model strumieniowania i buforowania danych	14
4.3.1	Konsekwencje braku sygnału <code>ready</code>	14
4.3.2	Buforowanie jako element architektury	15
4.3.3	Obsługa przerw w strumieniu wejściowym	15
4.3.4	Deterministyczność przepływu	16
4.4	Cykliczny kod nadmiarowy (CRC)	16
4.4.1	Wariant bazowy: moduł szeregowy <code>crc_serial</code>	18
4.4.2	Rozszerzenie modułu szeregowego o inicjalizację stanu CRC	18
4.4.3	Moduł <code>crc_update</code> : równoległa aktualizacja CRC o 64 bity	18
4.4.4	Generowanie równań XOR na podstawie macierzy H1 i H2	19
4.4.5	Moduł <code>crc_tb_parallel</code> : integracja trybu równoległego i szerego- wego	19
4.4.6	Dobór długości CRC na podstawie długości TB	20
4.4.7	Wyznaczenie końcówki TB (“ogona”) na podstawie <code>tb_len</code>	20
4.4.8	Tryb S_APPEND: ostatnie słowo pełne	20
4.4.9	Tryb S_SERIAL: ostatnie słowo niepełne	21
4.4.10	Podsumowanie roli CRC w torze	21
4.5	Przygotowanie danych do kodowania LDPC	21
4.5.1	Buforowanie TB i rozdzielenie faz pracy	22
4.5.2	Wyznaczenie parametrów CRC i wielkości pośrednich	23
4.5.3	Segmentacja TB na bloki kodowe (CB)	23
4.5.4	Dobór Z_c , wyznaczenie k oraz liczby <i>filler bits</i>	24
4.5.5	Formowanie strumienia wyjściowego CB na szynie 64-bit	25
4.5.6	Dopisanie CRC24B dla CB	25
4.5.7	Metadane konfiguracyjne przekazywane do kodera LDPC	26
4.6	Implementacja kodera LDPC	26
4.6.1	Interfejs wejściowy i buforowanie danych	26
4.6.2	ROM opisujący graf bazowy i dobór parametrów z tablic	27
4.6.3	Struktura obliczeń: okno Z_c , rotacja i akumulacja XOR	28
4.6.4	Realizacja przesunięcia cyklicznego dla bloku Z_c	29
4.6.5	Wyznaczanie bloków parzystości i obsługa przypadków szczególnych	29
4.6.6	Emisja danych: złączenie strumienia CB oraz metadane paddingu .	30
4.6.7	Załączki puncturingu i rate matchingu	32
4.7	Integracja top-level i sterowanie pracą toru	32

5 Weryfikacja i walidacja 35

5.1	Cele i kryteria poprawności	35
-----	---------------------------------------	----

5.1.1	Cele weryfikacji	35
5.1.2	Kryteria poprawności	35
5.2	Środowisko weryfikacji	36
5.2.1	Symulacja HDL (Questa)	36
5.2.2	Testbenche i automatyzacja przebiegu testów	37
5.2.3	Wektory referencyjne i narzędzia MATLAB	37
5.3	Organizacja danych testowych i wektorów referencyjnych	37
5.3.1	Struktura katalogów	38
5.3.2	Nazewnictwo plików i typy danych	38
5.3.3	Format plików bitowych	39
5.3.4	Format plików metadanych	39
5.4	Testy modułowe	39
5.4.1	Dane testowe, organizacja przypadków i format plików	40
5.4.2	Wspólny szkielet testbenchy i zasady porównywania	41
5.4.3	Test modułu <code>crc_serial</code>	41
5.4.4	Test modułów <code>crc_tb_parallel</code> oraz <code>crc_cb_parallel</code>	41
5.4.5	Test modułu <code>LDPC_prep</code>	42
5.4.6	Test modułu <code>LDPC_encode</code>	42
5.4.7	Test integracyjny modułu top-level <code>PDSCHTx</code>	42
5.5	Analiza wyników testów i symulacji	43
5.5.1	Definicje zdarzeń i metryk czasowych	43
5.5.2	Wybrane przypadki testowe i ich parametry	43
5.5.3	Latencja modułu <code>crc_serial</code>	43
5.5.4	Latencja modułu <code>crc_tb_parallel</code>	44
5.5.5	Latencja modułu <code>LDPC_prep</code>	44
5.5.6	Latencja modułu <code>LDPC_encode</code>	45
5.5.7	Latencja toru top-level <code>PDSCHTx</code>	45
5.5.8	Wnioski z analizy latencji	45
5.5.9	Narzut magistrali 64-bit	46
5.6	Wyniki syntezy i analiza implementacyjna w FPGA	47
5.6.1	Konfiguracja narzędzi i założenia kompilacji	47
5.6.2	Wykorzystanie zasobów układu	47
5.6.3	Analiza czasowa i maksymalna częstotliwość pracy	48
5.6.4	Wnioski z kompilacji	48
5.6.5	Szacowana przepustowość toru na podstawie $F_{\max}$ i latencji symulacyjnej	49
6	Podsumowanie i wnioski	51
	Bibliografia	53

Spis skrótów i symboli	57
Źródła	59
Lista dodatkowych plików, uzupełniających tekst pracy	61
Spis rysunków	63
Spis tabel	65

Rozdział 1

Wstęp

Rozdział 2

Analiza tematu

Rozdział 3

Wymagania i narzędzia

3.1 Założenia projektowe

3.1.1 Sposób przetwarzania danych

Podstawowym założeniem jest przetwarzanie danych w całym torze na stałej szynie danych o szerokości 64 bitów. Dane są przekazywane pomiędzy blokami jako kolejne słowa 64-bitowe, taktowane wspólnym sygnałem zegarowym `clk`. W konsekwencji:

- granice istotnych fragmentów danych nie muszą pokrywać się z granicami słowa 64-bit,
- w przypadku długości niebędącej wielokrotnością 64 bitów konieczne jest stosowanie dopełniania oraz przekazywanie informacji o ważności danych.

3.1.2 Model pracy

Aktualna wersja systemu obsługuje dokładnie jeden blok transportowy (TB) w ramach jednego cyklu pracy. Po zakończeniu przetwarzania TB nie przewidziano automatycznego powrotu do stanu gotowości. Aby rozpocząć przetwarzanie następnego bloku danych wymagane jest wykonanie resetu układu.

Założenie to upraszcza sterowanie modułami, zamykanie przetwarzania i obsługę buforów danych, kosztem braku pracy w trybie ciągłym. Rozszerzenie do obsługi transmisji ciągłej stanowi potencjalny kierunek rozwoju.

3.1.3 Sterowanie przepływem danych

W projekcie do kontroli przepływu danych zaimplementowano mechanizm opierający się na sygnałach `"valid"` oraz `"last"`. Mechanizm ten ideą przypomina protokół `"valid-ready"`.

- źródło danych sygnalizuje ważność słowa sygnałem `valid`,

- odbiornik nie posiada sygnału informującego o gotowości do przyjęcia danych (brak `ready`),
- odbiornik ma obowiązek przyjąć dane w każdym cyklu, w którym `valid=1`.

Przyjęcie takiego interfejsu upraszcza połączenia pomiędzy blokami oraz ogranicza liczbę przypadków brzegowych podczas pierwszej implementacji. Jednocześnie nakłada wymagania na strukturę toru. Moduły muszą przekazywać dane z wejścia na wyjście w cyklu ich wystąpienia albo posiadać odpowiednie buforu umożliwiające przyjęcie danych w każdym cyklu.

3.1.4 Sposób podawania danych wejściowych

Nie narzuca się stałego tempa dostarczania danych wejściowych. System nadrzędny może:

- podawać dane w sposób ciągły – do 1 słowa 64-bitowego na 1 cykl sygnału zegarowego,
- wprowadzać dowolne przerwy pomiędzy kolejnymi słowami.

Słowo jest uznawane za poprawne i podlegające przetwarzaniu wyłącznie w cyklu, w którym `in_valid=1`. Zakończenie TB jest sygnalizowane przez `in_last=1` jednocześnie z `in_valid=1` na ostatnim słowie TB.

Wprowadzenie pełnego mechanizmu handshake (np. zgodnego z AXI4-Stream) jest traktowane jako możliwy etap rozbudowy, lecz nie jest elementem aktualnej wersji projektu.

3.1.5 Elastyczność kodowania LDPC i obsługa różnych TB

Projekt zakłada obsługę dowolnego bloku transportowego (TB), tj. poprawne przetworzenie i zakodowanie danych niezależnie od długości TB oraz konfiguracji wynikającej z reguł doboru parametrów kodowania LDPC. Oznacza to, że tor przetwarzania powinien działać poprawnie dla różnych przypadków, w których mogą zmieniać się m.in.:

- wybór grafu bazowego (BG),
- wartość liftingu (Z_c),
- docelowe długości bloków używane w kodowaniu,
- liczba bitów dopełnienia oraz wynikające z niej wyrównanie danych,
- liczba bloków kodowych (CB) powstałych w wyniku segmentacji TB.

W konsekwencji architektura musi umożliwiać przetwarzanie bloków o zmiennej długości oraz generowanie poprawnego strumienia wyjściowego przy zachowaniu stałej szerokości słowa 64-bit.

3.1.6 Wymagania

- **Deterministyczność działania:** dla identycznych danych wejściowych oraz identycznych metadanych sterujących system powinien generować identyczny strumień wyjściowy.
- **Weryfikacja przez symulację:** projekt powinien umożliwiać weryfikację w symulacji poprzez porównanie wyników z wektorami referencyjnymi w sposób bitowo-dokładny.
- **Synteżowalność:** projekt powinien dać się poprawnie zsyntetyzować dla założonego układu FPGA, tak aby raporty syntezy pozwalały ocenić wykorzystanie zasobów oraz spełnienie ograniczeń czasowych.

3.2 Interfejsy i format danych

W projekcie zastosowano prosty interfejs strumieniowy oparty o sygnały `valid/last` oraz równoległą magistralę danych 64-bit. Interfejs ten jest używany zarówno na wejściu całego toru, jak i na wyjściu. Wewnątrz projektu poszczególne bloki mogą dodatkowo wymieniać metadane sterujące przetwarzaniem.

3.2.1 Interfejs wejściowy

Wejście projektu przyjmuje TB w postaci kolejnych słów 64-bitowych:

- `in_chunk[63:0]` – dane wejściowe,
- `in_valid` – informacja o tym, że `in_chunk` jest ważne w bieżącym cyklu,
- `in_last` – znacznik ostatniego słowa TB,
- `tb_len` – długość TB wyrażona w ilości bitów.

Bity w strumieniu są interpretowane w kolejności **MSB-first**, tj. `in_chunk[63]` odpowiada pierwszemu bitowi danego słowa w sensie strumieniowym, a `in_chunk[0]` – ostatniemu.

Interfejs wejściowy przedstawiono na rys. 3.1.

```
1      ...  
2      input  wire          clk ,  
3      input  wire          rst ,  
4      input  wire [63:0]    in_chunk ,  
5      input  wire          in_valid ,  
6      input  wire          in_last ,  
7      input  wire [15:0]    tb_len ,  
8      ...  
9 );
```

Rysunek 3.1: Interfejs wejściowy układu.

3.2.2 Interfejs wyjściowy

Wyjście projektu generuje strumień danych w postaci słów 64-bitowych:

- `out_chunk[63:0]` – dane wyjściowe,
- `out_valid` – informacja, że `out_chunk` jest ważne w bieżącym cyklu,
- `out_last` – znacznik ostatniego słowa,
- `out_pad_left[5:0]` – liczba bitów do pominięcia od strony MSB w słowie wyjściowym,
- `out_pad_right[5:0]` – liczba bitów do pominięcia od strony LSB w słowie wyjściowym.

Słowo wyjściowe jest uznawane za ważne wyłącznie w cyklu, w którym `out_valid=1`. Zakończenie przetwarzania całego TB jest jednoznacznie określone przez `out_last=1` podane wraz z ostatnim słowem wyjściowym.

Analogicznie do wejścia, bity w strumieniu wyjściowym są interpretowane w kolejności **MSB-first**, tj. `out_chunk[63]` odpowiada pierwszemu bitowi danego słowa w sensie strumieniowym, a `out_chunk[0]` – ostatniemu.

Sygnały `out_pad_left` oraz `out_pad_right` stanowią metadane umożliwiające jednoznaczną interpretację strumienia w przypadku, gdy liczba bitów danych nie jest wielokrotnością 64. Informują one, ile bitów w słowach należy pominąć odpowiednio od strony MSB oraz LSB.

Interfejs wyjściowy przedstawiono na rys. 3.2.

3.3 Środowisko projektowe i narzędzia

Do realizacji projektu wykorzystano zestaw narzędzi obejmujący środowisko do syntezy i implementacji układów FPGA, symulator HDL oraz środowisko do przygotowania i analizy danych referencyjnych.

```

1      ...
2      output wire [63:0]  out_chunk ,
3      output wire                out_valid ,
4      output wire                out_last ,
5      output wire [5:0]  out_pad_left ,
6      output wire [5:0]  out_pad_right
7 );

```

Rysunek 3.2: Interfejs wyjściowy układu.

3.3.1 Quartus Prime Lite Edition

Podstawowym narzędziem do syntezy, implementacji oraz analizy wykorzystania zasobów i parametrów czasowych jest Quartus Prime. Narzędzie to zostało użyte do:

- syntezy logiki opisanej w języku Verilog,
- implementacji (fitter) dla wybranego układu FPGA,
- generowania raportów wykorzystania zasobów,
- analizy czasowej (timing) oraz wyznaczenia maksymalnej częstotliwości pracy.

3.3.2 Questa Intel Starter FPGA Edition

Symulacje funkcjonalne projektu oraz uruchamianie środowiska testowego zrealizowano z wykorzystaniem symulatora Questa. Narzędzie to zostało użyte do:

- kompilacji i uruchamiania testbenchu HDL napisanych w języku System Verilog,
- analizy przebiegów czasowych oraz zawartości rejestrów w celu debugowania,
- weryfikacji poprawności działania poprzez porównanie wyników z danymi referencyjnymi.

3.3.3 MATLAB

Środowisko MATLAB wykorzystano do przygotowania i obsługi danych referencyjnych oraz do analizy wyników działania projektu. W szczególności MATLAB służył do:

- generowania wektorów testowych wejściowych i referencyjnych wyników wyjściowych,
- przygotowania metadanych testów,
- wstępnej analizy porównawczej wyników.

MATLAB został wykorzystany również do stworzenia kodu modelującego przetwarzanie strumienia. Kod ten umożliwiał wizualizację kolejnych transformacji wektorów i weryfikację oczekiwanego formatu danych na wyjściu poszczególnych etapów toru.

Rozdział 4

Koncepcja i realizacja projektu

4.1 Założenia architektoniczne toru

Architektura toru została zaprojektowana jako rozwiązanie strumieniowe, którego podstawowym zadaniem jest przetwarzanie danych TB przy zachowaniu stałej szerokości słowa 64-bit. Decyzje architektoniczne wynikają bezpośrednio z przyjętych założeń projektowych (rozdz. 3) oraz z charakterystyki kodowania LDPC, w którym parametry kodowania mogą się zmieniać w zależności od długości TB.

4.1.1 Stała magistrala 64-bit jako oś projektu

Cały tor oparto o jednolity format przesyłania danych: słowa 64-bitowe w kolejności MSB-first. Ujednolicenie szerokości interfejsu upraszcza integrację bloków i umożliwia utrzymanie wysokiej przepustowości, jednak wprowadza istotne konsekwencje:

- granice logiczne (koniec TB, granice CB, granice części systematycznej i parzystości) mogą wypadać wewnątrz słowa 64-bit,
- konieczna jest jawna obsługa wyrównania i dopełniania oraz przenoszenie informacji umożliwiającej odtworzenie dokładnej liczby bitów danych.

Z tego względu w projekcie przewidziano mechanizmy pozwalające na jednoznaczną interpretację strumienia w obecności paddingu.

4.1.2 Uproszczony model sterowania strumieniem

W celu ograniczenia złożoności pierwszej wersji projektu przyjęto jednokierunkowy handshake oparty o sygnały `valid/last`, bez sygnału `ready`. Oznacza to, że blok odbierający dane ma obowiązek ich przyjęcia w każdym cyklu z `valid=1`. Taki model upraszcza połączenia pomiędzy etapami toru, ale jednocześnie wymusza odpowiednie podejście architektoniczne:

- etapy przetwarzające dane „w locie” muszą być przeźroczyste w sensie strumienia (brak możliwości zatrzymania źródła),
- etapy wymagające dostępu do większej ilości danych (np. operacje blokowe) muszą posiadać buforowanie, aby nie utracić danych wejściowych.

W projekcie dopuszczono zarówno strumień ciągły (1 słowo/cykl), jak i wejście z przerwami (`valid=0`), co zapewnia swobodę po stronie źródła danych.

4.1.3 Przetwarzanie blokowe i reset jako granica transakcji

Aktualna wersja obsługuje pojedynczy TB pomiędzy resetami. Z punktu widzenia architektury reset pełni rolę granicy transakcji: po resecie układ przechodzi do stanu początkowego, zbiera dane TB, wykonuje pełne przetwarzanie, a następnie kończy pracę na `out_last`. Takie podejście upraszcza sterowanie i zamykanie przetwarzania, a także ułatwia weryfikację symulacyjną poprzez jednoznaczne odseparowanie przypadków testowych.

4.1.4 Elastyczność kodowania LDPC

Kodowanie LDPC w 5G NR może występować w wielu konfiguracjach. W związku z tym tor zaprojektowano jako konfigurowalny, tzn. zdolny do poprawnego przetwarzania TB dla różnych zestawów parametrów w zakresie wspieranym przez implementację. Konsekwencją tej elastyczności jest konieczność:

- przenoszenia metadanych konfiguracyjnych pomiędzy etapami toru,
- obsługi zmiennych długości bloków i zmiennej liczby słów w strumieniu wyjściowym,
- utrzymania spójnego formatu danych niezależnie od wybranej konfiguracji.

4.1.5 Weryfikacja poprzez symulację i synteżowalność

Rozwiązanie opracowano tak, aby umożliwiało weryfikację bitowo-dokładną w symulacji z wykorzystaniem wektorów referencyjnych oraz aby było poprawnie synteżowalne dla zadanego układu FPGA. Raporty syntezy stanowią podstawę oceny wykorzystania zasobów i parametrów czasowych, co pozwala formalnie ocenić wykonalność projektu w warunkach narzuconych przez narzędzie syntezy.

4.2 Struktura funkcjonalna i przepływ danych

Przyjęta architektura stanowi tor przetwarzania danych, w którym wejściowy TB jest przekształcany do postaci strumienia danych wyjściowych po operacjach przygotowania

i kodowania kanałowego. Tor ma charakter sekwencyjny: dane przechodzą przez kolejne etapy, a każdy etap realizuje określoną funkcję w procesie wytwarzania końcowego strumienia.

4.2.1 Ogólny podział toru na etapy

Z punktu widzenia funkcjonalnego tor można podzielić na następujące etapy:

1. przyjęcie danych TB w strumieniu 64-bit, wraz z długością,
2. obliczenie i dołączenie sumy kontrolnej CRC,
3. przygotowanie danych do kodowania LDPC:
 - (a) obliczenie parametrów kodowania na podstawie długości TB,
 - (b) segmentacja TB na bloki kodowe (CB),
 - (c) obliczenie i dołączenie sumy kontrolnej CRC do każdego CB.
4. kodowanie LDPC i wygenerowanie danych wyjściowych.

4.2.2 Przepływ danych wejściowych i moment rozpoczęcia przetwarzania

Dane wejściowe są dostarczane jako kolejne słowa 64-bitowe. Strumień może zawierać przerwy (cykle z `in_valid=0`), a system rozpoznaje koniec TB na podstawie `in_last=1` podanego wraz z ostatnim ważnym słowem.

Ze względu na obecność etapów o charakterze blokowym (w szczególności operacji zależnych od pełnej długości TB oraz parametrów kodowania), w torze występuje faza zbierania danych, następnie przetwarzania, a na końcu generowania danych wyjściowych. Dokładny moment rozpoczęcia emisji wyjścia zależy od długości TB, a emisja danych wyjściowych nie może rozpocząć się przed odebraniem całej wiadomości na wejściu.

4.2.3 Przepływ danych wewnątrz toru

Miedzy etapami przetwarzania dane są przekazywane jako strumień 64-bitowy z semantyką `valid/last`. Wewnętrzne połączenia mogą dodatkowo przenosić metadane niezbędne do konfiguracji kodowania, w szczególności takie, które zależą od długości TB i wynikowego doboru parametrów LDPC.

Ważną cechą toru jest to, że część etapów musi operować na danych o zmiennej długości (np. bloki o długości zależnej od Z_c), podczas gdy interfejs transportuje dane w stałych słowach 64-bit. Z tego względu na granicach etapów występują operacje składania/rozbijania danych bitowych oraz mechanizmy wyrównania.

4.2.4 Strumień wyjściowy i zakończenie przetwarzania

Efektem działania toru jest strumień wyjściowy w postaci słów 64-bitowych `out_chunk`, oznaczonych `out_valid`. Zakończenie przetwarzania całego TB jest sygnalizowane przez `out_last=1` podane wraz z ostatnim słowem wyjściowym.

Ze względu na stałą szerokość słowa, strumień wyjściowy może zawierać bity dopełnienia w słowach brzegowych. Projekt udostępnia metadane umożliwiające jednoznaczną interpretację tych przypadków, sygnały `out_pad_left` oraz `out_pad_right` informują, które bity słowa są ważne.

4.2.5 Warianty długości TB i konsekwencje dla przepływu danych

Architektura zakłada poprawną obsługę TB o różnych długościach. Zmiana długości TB wpływa na:

- liczbę słów wejściowych w strumieniu,
- dobór parametrów kodowania LDPC,
- liczbę oraz rozmiar przetwarzanych bloków kodowych (CB),
- liczbę słów w strumieniu wyjściowym oraz wielkość paddingu w słowach brzegowych.

W konsekwencji tor musi być przygotowany na to, że dla różnych przypadków testowych czas trwania przetwarzania oraz długość strumienia będą różne, mimo identycznego formatu interfejsu.

4.3 Model strumieniowania i buforowania danych

Tor przetwarzania oparto o interfejs strumieniowy z sygnałami `valid/last` bez sygnału `ready`. Oznacza to, że dane są przekazywane w sposób jednokierunkowy: źródło sygnalizuje ważność słowa, a odbiornik ma obowiązek je przyjąć. Taki model wpływa bezpośrednio na sposób organizacji modułów, konieczność buforowania oraz sposób obsługi przerw w strumieniu wejściowym.

4.3.1 Konsekwencje braku sygnału `ready`

Brak `ready` oznacza, że blok odbiorczy nie może wymusić wstrzymania źródła danych. Z punktu widzenia architektury wymusza to spełnienie dwóch warunków:

- każdy etap toru musi być w stanie przyjąć dane zawsze, gdy poprzedni etap zgłasza `valid=1`,

- jeżeli etap nie może przetworzyć danych w locie, musi posiadać bufor umożliwiający ich zapis i późniejsze przetwarzanie.

W praktyce oznacza to, że moduły w torze dzielą się na:

- **moduły przeźroczyste strumieniowo** – przekazujące dane dalej w tym samym cyklu (lub z ustalonym, stałym opóźnieniem) przy zachowaniu semantyki `valid/last`,
- **moduły blokowe** – wymagające zebrania większej ilości danych i dlatego wykorzystują buforowanie w pamięci wewnętrznej.

4.3.2 Buforowanie jako element architektury

Zastosowanie buforów wynika z charakterystyki przetwarzania: część operacji wymaga dostępu do danych w sposób niedostępny w czysto strumieniowym przetwarzaniu 1:1. Buforowanie w projekcie pełni następujące role:

- umożliwia zebranie danych wymaganych do wyznaczenia parametrów przetwarzania,
- umożliwia dostęp do wcześniej odebranych danych podczas formowania bloków o zmiennej długości,
- umożliwia obliczenia wykonywane wielokrotnie na tych samych danych.

4.3.3 Obsługa przerw w strumieniu wejściowym

Źródło danych może wprowadzać przerwy pomiędzy kolejnymi słowami TB, tj. występują cykle, w których `in_valid=0`. W takim przypadku:

- układ nie próbuje próbować `in_chunk` ani nie zmienia stanu buforów danych,
- licznik odebranych słów/bitów jest zwiększany wyłącznie w cyklach, w których `in_valid=1`,
- koniec TB jest rozpoznawany dopiero w cyklu, w którym równocześnie `in_valid=1` oraz `in_last=1`.

Dzięki temu zachowanie toru nie zależy od tempa dostarczania danych, a jedynie od ich kolejności oraz sygnałów sterujących.

4.3.4 Deterministyczność przepływu

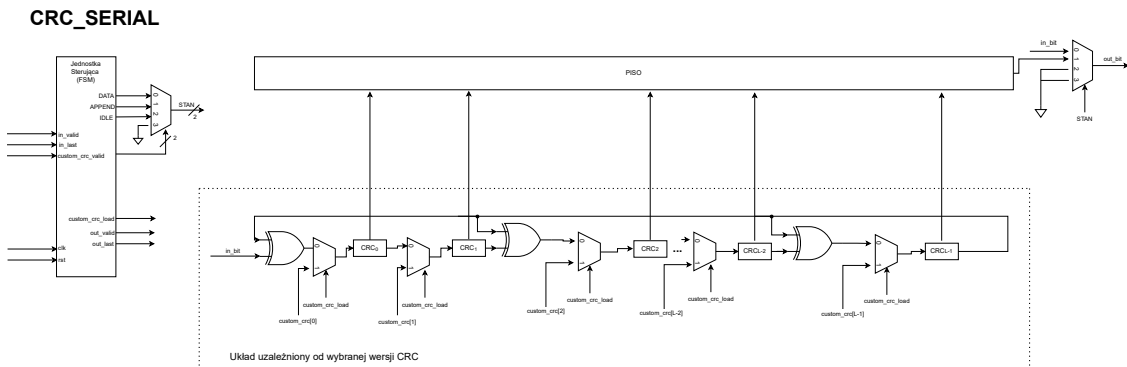
Model `valid/last` bez `ready` oraz jawne buforowanie powodują, że przepływ danych jest deterministyczny: kolejność słów w buforach oraz kolejność emisji na wyjściu zależą wyłącznie od kolejności nadejścia danych oraz metadanych. W połączeniu z weryfikacją symulacyjną umożliwia to jednoznaczne porównywanie strumienia wyjściowego z wektorami referencyjnymi.

4.4 Cykliczny kod nadmiarowy (CRC)

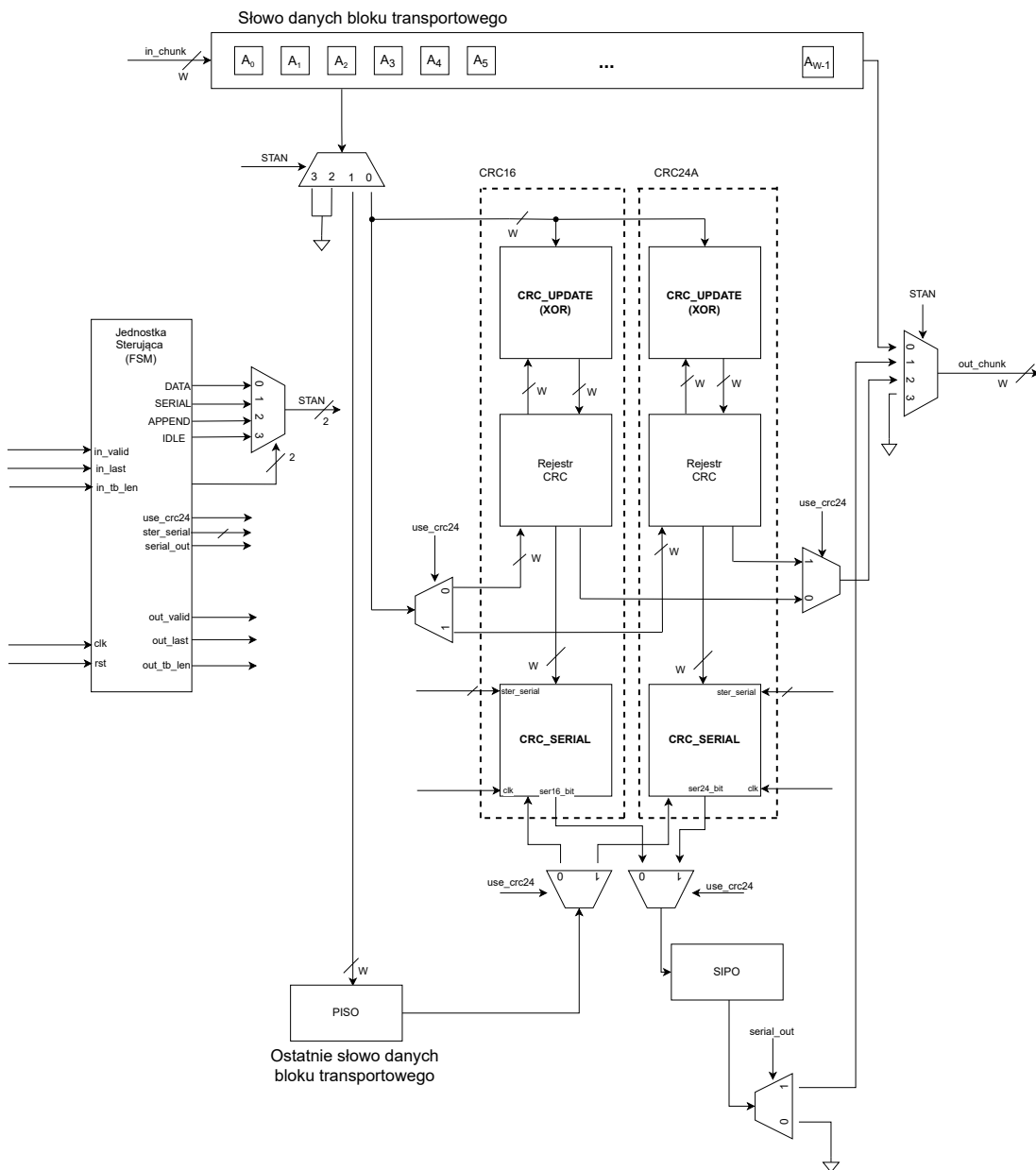
W torze nadawczym 5G NR suma kontrolna CRC jest dopisywana do bloku transportowego (TB) przed dalszymi etapami przetwarzania. W projekcie zastosowano dwustopniowe podejście do realizacji CRC:

- najpierw opracowano moduł szeregowy działający bit-po-bicie (`crc_serial`),
- następnie rozbudowano rozwiązanie do postaci równoległej 64-bit (`crc_tb_parallel`), wykorzystując kombinacyjną aktualizację CRC o całe słowo (`crc_update`) oraz ponownie używając `crc_serial` do obsługi niepełnej końcówki TB.

Schemat blokowy rozwiązania przedstawiono na rys. 4.1 oraz 4.2.



Rysunek 4.1: Schemat blokowy modułu `crc_serial` liczącego CRC szeregowo.

CRC_TB/CB_PARALLELRysunek 4.2: Schemat blokowy modułu `crc_tb_parallel` liczącego CRC równolegle.

4.4.1 Wariant bazowy: moduł szeregowy `crc_serial`

Moduł `crc_serial` realizuje obliczanie CRC w sposób klasyczny, tj. aktualizuje rejestr CRC o jeden bit na cykl zegara i jednocześnie przepuszcza strumień danych na wyjście. Po zakończeniu danych wejściowych moduł dopisuje na wyjściu kolejne bity CRC, a sygnał `out_last` występuje tylko raz – na ostatnim bicie CRC. Interfejs modułu jest bitowy (1 bit/cykl) i wykorzystuje sterowanie `valid/last`.

Wewnątrz `crc_serial` zastosowano rejestr CRC o długości zależnej od parametru `L`. W aktualnej wersji przewidziano wszystkie warianty używane w standardzie 5G NR:

- CRC16 – używany dla TB o długości mniejszej lub równej 3824 bitów,
- CRC24A – używany dla TB o długości większej niż 3824 bity,
- CRC24B – używany dla CB.

4.4.2 Rozszerzenie modułu szeregowego o inicjalizację stanu CRC

W praktycznej integracji CRC w torze równoległym pojawia się potrzeba podania stanu początkowego rejestru CRC, który odpowiada obliczeniom wykonanym wcześniej na pełnych słowach 64-bit. Z tego powodu moduł `crc_serial` został rozszerzony o wejścia:

- `custom_crc` — wartość stanu CRC, od której ma rozpocząć się dalsze liczenie,
- `custom_crc_valid` — sygnał wyboru (gdy aktywny, stan startowy nie jest zerowany, tylko ładowany z `custom_crc`).

Rozszerzenie to umożliwia użycie `crc_serial` jako kontynuacji obliczeń w sytuacji, gdy część TB została już przetworzona równolegle, a pozostała jedynie niepełna końcówka wymagająca operacji bitowych.

4.4.3 Moduł `crc_update`: równoległa aktualizacja CRC o 64 bity

Aby umożliwić pracę w torze 64-bitowym bez iteracji bit-po-bicie dla każdego słowa, zastosowano moduł `crc_update`. Jest to blok kombinacyjny, który wyznacza:

$$CRC_{next} = f(CRC_{in}, data[63 : 0])$$

gdzie funkcja $f(\cdot)$ jest zależnością liniową w ciele $GF(2)$ (realizowaną jako XOR wybranych bitów stanu i danych).

Podejście to odpowiada koncepcji generatorów CRC równoległych, w których stan następny jest funkcją aktualnego stanu oraz równoległego wektora danych [1].

4.4.4 Generowanie równań XOR na podstawie macierzy $H1$ i $H2$

Równoległą aktualizację CRC można wyznaczyć z własności liniowości obliczeń w $GF(2)$. Najpierw buduje się dwie macierze zależności:

- $H1$ opisuje, jak każdy bit wejściowego słowa danych wpływa na poszczególne bity CRC na wyjściu przy zerowym stanie początkowym CRC,
- $H2$ opisuje, jak każdy bit początkowego stanu CRC wpływa na poszczególne bity CRC na wyjściu przy słowie danych składającym się z samych zer.

Następnie dla każdego bitu CRC na wyjściu odczytuje się z tych macierzy, które wejścia mają na niego wpływ. W praktyce wygląda to tak, że:

- jedynki w odpowiedniej kolumnie $H1$ wskazują, które bity danych należy zsumować operacją XOR,
- jedynki w odpowiedniej kolumnie $H2$ wskazują, które bity bieżącego stanu CRC należy również dołączyć do XOR.

Ostatecznie każdy bit jest XOR-em wybranych bitów danych oraz wybranych bitów stanu CRC. Zestaw takich równań stanowi gotową postać kombinacyjną aktualizacji CRC o całe słowo danych w jednym kroku.

W projekcie zastosowano skrypt MATLAB:

- `gen_parallel_crc_matrix.m` — wyznacza macierze $H1/H2$ dla zadanych wielomianów (CRC16, CRC24A, CRC24B) i generuje linie równań XOR w postaci nadającej się do przeniesienia do kodu HDL.

W efekcie moduł `crc_update` zawiera funkcje (dla odpowiednich wariantów CRC), w których każda linia ma postać:

$$crc_out[i] = \bigoplus (\text{wybrane bity } crc_in \text{ oraz } data)$$

co odpowiada dokładnie 64 krokom aktualizacji szeregowej wykonanym w jednym cyklu.

4.4.5 Moduł `crc_tb_parallel`: integracja trybu równoległego i szeregowego

Moduł `crc_tb_parallel` stanowi właściwą implementację CRC w torze 64-bitowym. Dla kolejnych słów wejściowych:

- przekazuje dane na wyjście 1:1,

- równolegle aktualizuje rejestr CRC o 64 bity na podstawie `crc_update`.

Po wykryciu końca TB (`in_last=1`) moduł przechodzi do dopisania CRC. W zależności od tego, czy ostatnie słowo TB jest pełne, stosowane są dwa tryby pracy (zgodne z ideą pokazaną na rys. 4.2).

4.4.6 Dobór długości CRC na podstawie długości TB

Wybór wariantu CRC jest realizowany na podstawie długości TB:

- dla `in_tb_len > 3824` wybierany jest wariant CRC24A,
- dla `in_tb_len <= 3824` wybierany jest wariant CRC16.

4.4.7 Wyznaczenie końcówki TB (“ogona”) na podstawie `tb_len`

Istotnym założeniem projektu jest to, że na wejściu nie jest podawana informacja o dopełnieniu ostatniego słowa (padding). Zamiast tego moduł wykorzystuje `tb_len`, aby wyznaczyć liczbę ważnych bitów w ostatnim słowie TB:

- obliczane jest `rem = tb_len mod 64`,
- jeżeli `rem=0`, ostatnie słowo ma 64 ważne bity,
- w przeciwnym razie ostatnie słowo ma `rem` ważnych bitów.

Dzięki temu wystarczające jest przekazanie samej długości TB w bitach, a źródło danych nie musi raportować osobno ile bitów w ostatnim słowie jest ważnych. Z tej samej przyczyny na wyjściu modułu CRC nie jest przekazywana informacja o paddingu ostatniego słowa – długość TB pozostaje znana i niezmienna (moduł przekazuje `out_tb_len` bez CRC jako metadane ramki).

4.4.8 Tryb `S_APPEND`: ostatnie słowo pełne

Jeżeli ostatnie słowo TB jest pełne (64 ważne bity), to po jego wypuszczeniu moduł:

- emituje osobne słowo 64-bit zawierające CRC umieszczone w najbardziej znaczących bitach,
- pozostałe bity słowa CRC są zerowane,
- `out_last` jest ustawiane przy tym słowie CRC (kończąc całą ramkę).

Takie rozwiązanie upraszcza dopisanie CRC, ponieważ nie jest wymagane mieszanie końcówki danych z CRC w jednym słowie.

4.4.9 Tryb S_SERIAL: ostatnie słowo niepełne

Jeżeli ostatnie słowo TB jest niepełne, konieczne jest potraktowanie pozostałych bitów końcówki jako strumienia bitowego i dopisanie CRC w sposób zapewniający poprawną kolejność bitów oraz wyrównanie do granicy 64 bitów.

W tym celu `crc_tb_parallel`:

- przełącza się na ścieżkę bitową,
- uruchamia `crc_serial` jako moduł pomocniczy,
- ładuje do `crc_serial` stan CRC wyliczony wcześniej na pełnych słowach 64-bit (sygnały `custom_crc/custom_crc_valid`),
- podaje do `crc_serial` kolejne bity ostatniego słowa (MSB-first) i oznacza ostatni bit danych zgodnie z wyliczoną liczbą ważnych bitów,
- odbiera strumień bitowy zawierający końcówkę danych + CRC i składa go z powrotem do słów 64-bit.

Jeżeli suma liczby bitów (końcówka danych + CRC) przekracza 64, wyjście tworzy dwa słowa 64-bit (wyrównanie do 128 bitów). W przeciwnym wypadku powstaje jedno słowo wyjściowe, dopełnione zerami do pełnych 64 bitów.

4.4.10 Podsumowanie roli CRC w torze

Zastosowana architektura CRC łączy prostotę implementacji szeregowej z wydajnością przetwarzania równoległego:

- pełne słowa 64-bit przetwarzane są w jednym cyklu przez `crc_update`,
- wyłącznie niepełna końcówka TB wymaga pracy bitowej, zrealizowanej przez ponowne użycie `crc_serial`,
- `tb_len` eliminuje potrzebę przekazywania informacji o paddingu na wejściu.

To podejście pozwala zachować spójny format danych w całym torze przy jednoczesnym zachowaniu bitowo-dokładnej zgodności z referencją.

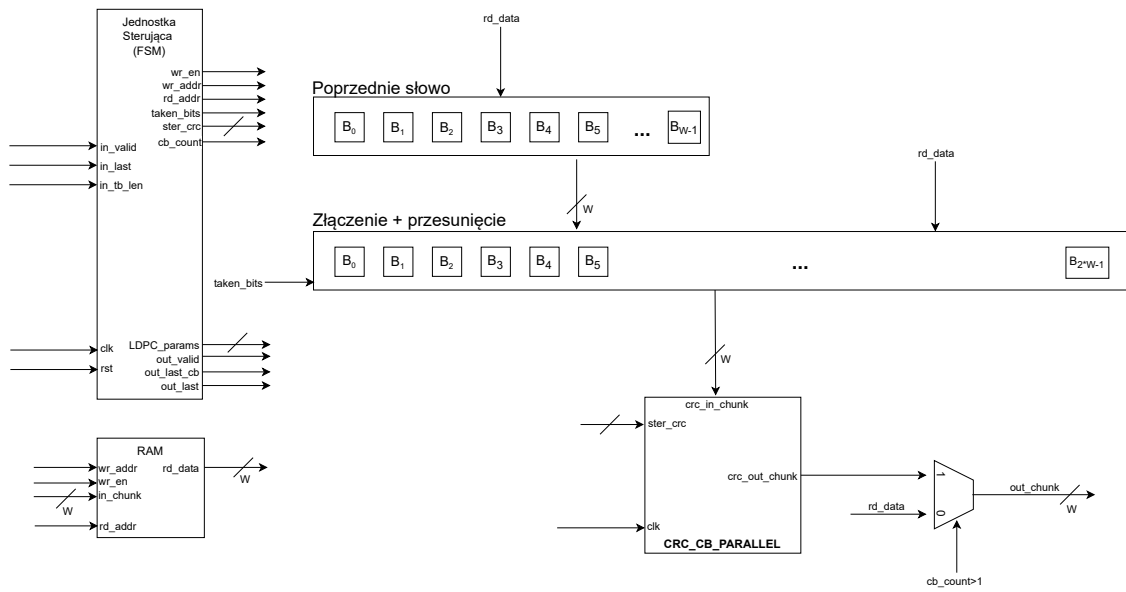
4.5 Przygotowanie danych do kodowania LDPC

Etap przygotowania danych do kodowania LDPC realizuje moduł `LDPC_prep`. Jego zadaniem jest przekształcenie wejściowego strumienia TB do postaci sekwencji bloków kodowych (CB) gotowych do podania na wejście kodera LDPC wraz z kompletem metadanych konfiguracyjnych (BG, Z_c , k , liczba filler bits). Moduł odpowiada za wyznaczenie

parametrów wynikających ze standardowych reguł, segmentację TB na CB, obliczenie i dopisanie CRC (jeśli wymagane), oraz przygotowanie strumienia wyjściowego przy zachowaniu stałej szerokości 64 bitów.

Schemat blokowy rozwiązania przedstawiono na rys. 4.3.

LDPC_PREP



Rysunek 4.3: Schemat blokowy modułu LDPC_prep przygotowującego dane do kodowania LDPC.

4.5.1 Buforowanie TB i rozdzielenie faz pracy

Ponieważ przygotowanie CB wymaga operowania na danych o zmiennej długości oraz wyznaczenia parametrów zależnych od całej ramki, moduł LDPC_prep w pierwszym kroku buforuje cały TB w pamięci wewnętrznej słowami 64-bit. Dane są zapisywane wyłącznie w cyklach, w których $in_valid=1$, a koniec TB jest wykrywany poprzez $in_last=1$ podane razem z ostatnim ważnym słowem. Przerwy w strumieniu wejściowym ($in_valid=0$) nie wpływają na poprawność działania — powodują jedynie wydłużenie fazy odbioru.

W konsekwencji praca modułu jest naturalnie podzielona na trzy fazy:

- **faza odbioru** – zapis TB do pamięci RAM,
- **faza przygotowania** – obliczenie parametrów segmentacji i LDPC,

- **faza segmentacji i emisji** – odczytanie odpowiednich fragmentów TB, dopisanie CRC (jeśli wymagane) oraz emisja bloków kodowych jako strumień 64-bit.

4.5.2 Wyznaczenie parametrów CRC i wielkości pośrednich

Na podstawie długości `tb_len` wyznaczana jest długość CRC dla TB (16 lub 24 bity) oraz wielkość pośrednia:

$$B = TB + CRC_{TB}$$

czyli liczba bitów po dopisaniu CRC dla TB. Informacja `tb_len` jest wystarczająca do dalszych obliczeń i do wyznaczania końcówek (bitów niepełnych słów) — nie jest wymagane przekazywanie oddzielnej informacji o paddingu ostatniego słowa na wejściu.

4.5.3 Segmentacja TB na bloki kodowe (CB)

Jeżeli długość B przekracza dopuszczalny rozmiar pojedynczego CB dla wybranego grafu bazowego, moduł wyznacza liczbę bloków kodowych C oraz długości pośrednie używane do przygotowania CB. W przypadku segmentacji uwzględniany jest narzut CRC dla CB (CRC24B), co powoduje powstanie wielkości:

$$B' = B + C \cdot CRC_{CB}$$

Następnie wyznaczana jest ilość danych przypadającej na jeden CB. Wartość k_{CB} zależy od tego, czy występuje segmentacja:

$$k_{CB} = \begin{cases} B, & C = 1 \\ \left\lceil \frac{B'}{C} \right\rceil, & C > 1 \end{cases}$$

gdzie B oznacza długość bloku po dopisaniu CRC dla TB, natomiast $B' = B + C \cdot CRC_{CB}$ uwzględnia dodatkowy narzut CRC24B dopisywany do każdego CB w przypadku segmentacji.

W projekcie obliczenia wymagające dzielenia z zaokrągleniem w górę (np. dla C oraz długości danych CB) realizowane są sprzętowo przez pomocniczy moduł `Ceil`. Realizuje on dzielenie w sposób iteracyjny, metodą kolejnych odejmowań. Po uruchomieniu wartości licznika i mianownika zatraskiwane są do rejestrów roboczych, a następnie w każdym cyklu odejmuje mianownik od bieżącej wartości licznika i jednocześnie inkrementuje licznik ilorazu. Iteracje trwają aż do momentu, gdy pozostała wartość jest mniejsza od mianownika – wtedy moduł przechodzi do stanu wyniku, zapamiętuje resztę i wystawia iloraz jako liczbę wykonanych odejmowań. Sygnał `result_valid` informuje o gotowości wyniku; wynik jest podawany tak długo, jak utrzymywane jest `enable=1`, a zwolnienie

`enable` powoduje powrót do stanu bezczynności i gotowość do kolejnego obliczenia. Z uwagi na iteracyjny charakter (jedno odejmowanie na cykl) moduł ma zmienną latencję zależną od wartości ilorazu.

Uwaga projektowa: w aktualnej wersji dobór grafu bazowego (BG) jest podyktowany wyłącznie długością bloku transportowego. Jest to świadome uproszczenie interfejsu, ponieważ do pełnego doboru BG w 5G NR potrzebne są dodatkowe parametry, które nie są dostarczane na wejściu modułu, ponieważ nie są one częścią tej implementacji. W praktyce dobór BG może być realizowany przez nadrzędny system sterujący, który zna pełen kontekst transmisji.

4.5.4 Dobór Z_c , wyznaczenie k oraz liczby *filler bits*

Po wyznaczeniu długości danych przypadającej na pojedynczy CB moduł określa parametry wymagane do skonfigurowania kodera LDPC. W tym miejscu istotne jest rozróżnienie dwóch pojęć, które w bywają opisywane podobnymi symbolami, natomiast w implementacji pełnią różne role:

- k_b (**zmienne**) – liczba „bloków danych” potrzebnych do przeniesienia aktualnej długości informacji w CB; wartość ta zależy od wybranego grafu bazowego oraz od długości danych. Jest ona wykorzystywana do wyznaczenia minimalnej wymaganej wartości Z_c .
- K_b (**stałe dla grafu**) – docelowa liczba bloków danych wynikająca bezpośrednio z grafu bazowego. Dla BG1 jest ona stała i wynosi $K_b = 22$, natomiast dla BG2 wynosi $K_b = 10$.

Najpierw wyznaczana jest minimalna wartość Z_c zapewniająca możliwość umieszczenia danych w CB przy zadanym k_b :

$$Z_{c,\min} = \left\lceil \frac{k_{CB}}{k_b} \right\rceil$$

a następnie dobierane jest docelowe Z_c z listy dopuszczalnych wartości (co zapewnia zgodność ze zdefiniowanymi w standardzie wartościami liftingu).

Po ustaleniu Z_c wyznaczana jest docelowa długość części systematycznej kodu, wynikająca ze struktury grafu bazowego:

$$k = Z_c \cdot K_b$$

gdzie K_b jest stałe dla danego BG. Oznacza to, że w przypadku BG2 nawet wtedy, gdy z obliczeń wynika $k_b < 10$, długość k pozostaje równa $Z_c \cdot 10$, a „brakujące” bloki danych są wypełniane bitami typu filler.

Liczba bitów dopełniających (*filler bits*) jest wyznaczana jako:

$$N_{\text{filler}} = k - k_{CB}$$

i w module jest udostępniana na wyjściu jako `out_filler_cnt`. Wartość ta określa, ile bitów w części systematycznej CB nie niesie informacji użytkownika i powinno zostać wypełnionych zgodnie z przyjętą w torze konwencją filler.

4.5.5 Formowanie strumienia wyjściowego CB na szynie 64-bit

Wyjście `LDPC_prep` stanowi strumień słów 64-bit zawierających dane kolejnych bloków kodowych. Moduł zapewnia poprawne przejścia granic bitowych CB nawet wtedy, gdy granice CB wypadają wewnątrz słowa 64-bit w buforze TB. W tym celu wykorzystywane jest składanie słów na podstawie pary kolejnych słów bufora oraz przesunięcia bitowego, tak aby uzyskać ciągły strumień bitów CB w konwencji MSB-first.

W zależności od liczby bloków kodowych występują dwa przypadki:

- **Pojedynczy CB** ($C = 1$): moduł emituje zawartość bufora TB słowo-po-słowie. W tym wariancie nie występuje CRC dla CB (zgodnie z regułą, że CRC24B jest stosowane dopiero przy segmentacji).
- **Wiele CB** ($C > 1$): dla każdego CB moduł pobiera z bufora odpowiednią liczbę bitów danych, a następnie dopisuje CRC24B dla CB i emituje wynik jako strumień 64-bit.

Granice bloków CB na wyjściu są sygnalizowane przez:

- `out_last_chunk` — znacznik ostatniego słowa bieżącego CB,
- `out_last_cb` — znacznik ostatniego CB w całym TB.

4.5.6 Dopisanie CRC24B dla CB

Dla przypadku segmentacji ($C > 1$) moduł dopisuje CRC dla każdego bloku kodowego (CRC24B) przy użyciu bloku `crc_cb_parallel`. Funkcjonalnie moduł ten jest odpowiednikiem `crc_tb_parallel` (opisanym w sekcji 4.4) — pracuje na szynie 64-bit i realizuje dopisanie CRC do strumienia danych, z obsługą sytuacji, gdy końcówka danych CB nie kończy się na granicy słowa 64-bit. Z tego względu w niniejszym rozdziale nie powiela się pełnego opisu wewnętrznej struktury CRC dla CB; można odwołać się do schematu i mechanizmów przedstawionych w sekcji 4.4.

4.5.7 Metadane konfiguracyjne przekazywane do kodera LDPC

Wraz z danymi CB moduł generuje metadane wymagane do skonfigurowania kodera LDPC:

- `out_k` — docelowa długość bloku kodowego po dopełnieniu,
- `out_zc` — dobrana wartość Z_c ,
- `out_bg` — wybrany graf bazowy (1 oznacza graf bazowy 1, 0 oznacza graf bazowy 2),
- `out_filler_cnt` — liczba bitów dopełniających (filler).

4.6 Implementacja kodera LDPC

Najważniejszym blokiem toru jest moduł `LDPC_encode`, realizujący kodowanie QC-LDPC na poziomie pojedynczych bloków kodowych (CB) w stałym formacie strumieniowym 64-bit. Moduł został zaprojektowany jako rozwiązanie elastyczne: dla zadanych parametrów (BG , Z_c , k , *filler bits*) potrafi zakodować CB zgodnie z regułami grafów bazowych 5G NR oraz wygenerować strumień bitów kodowych, zachowując 64-bitową magistralę danych na wejściu i wyjściu.

Schemat blokowy kodera przedstawiono na rys. 4.4.

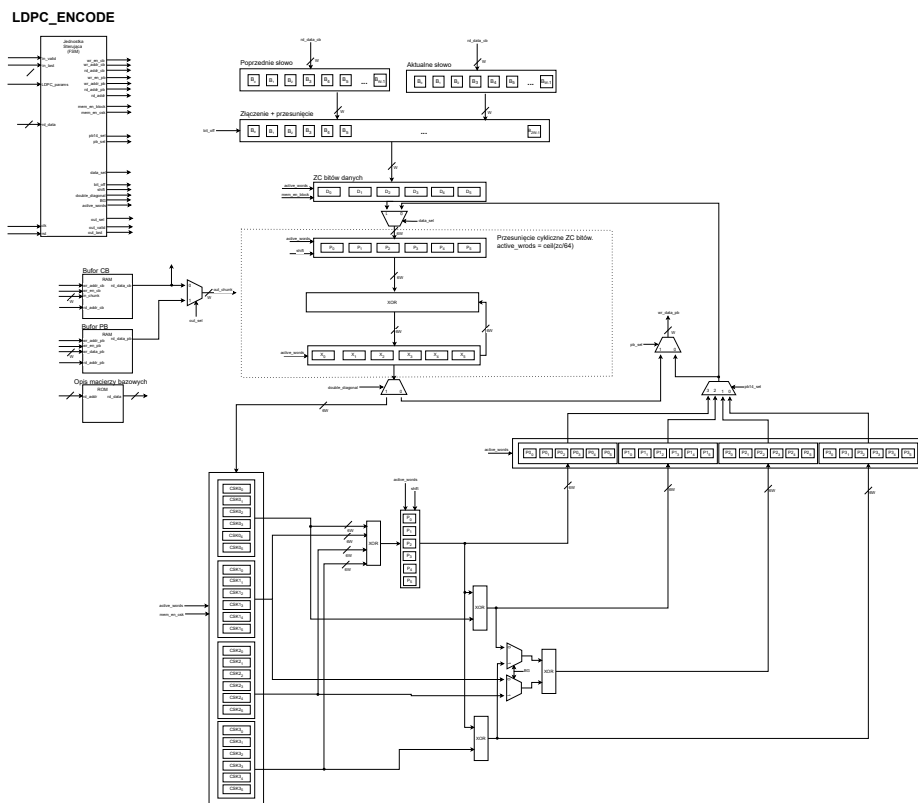
4.6.1 Interfejs wejściowy i buforowanie danych

Wejście kodera stanowi strumień słów 64-bit sterowany sygnałami `in_valid/in_last` oraz metadanymi konfiguracyjnymi:

- `in_k` – docelowa długość części systematycznej k dla CB,
- `in_zc` – wartość liftingu Z_c ,
- `in_bg` – wybór grafu bazowego (BG1/BG2),
- `in_filler_cnt` – liczba filler bits w CB.

Z uwagi na brak sygnału `ready`, przyjęto model, w którym dane są próbkowane zawsze, gdy `in_valid=1`, a źródło ma obowiązek podać stabilne `in_chunk` w cyklu ważności. Moduł dopuszcza przerwy w strumieniu (`in_valid=0`) – skutkują one jedynie wydłużeniem czasu buforowania.

W pierwszej fazie pracy koder buforuje cały CB w pamięci wewnętrznej słowami 64-bit. Granice CB są wyznaczone przez `in_last=1` (ostatnie słowo danych danego CB), natomiast `in_last_cb=1` oznacza ostatni CB w ramach przetwarzanego TB. W trakcie



Rysunek 4.4: Schemat blokowy modułu LDPC_encode.

buforowania zapamiętywane są adresy pierwszego i ostatniego słowa każdego CB oraz liczba słów, co pozwala później odwoływać się do danych CB w sposób losowy podczas obliczeń parzystości.

4.6.2 ROM opisujący graf bazowy i dobór parametrów z tablic

Kodowanie QC-LDPC w 5G NR jest zdefiniowane przez graf bazowy (BG1 lub BG2) oraz lifting Z_c , które determinują strukturę macierzy kontrolnej parzystości w postaci macierzy blokowej $Z_c \times Z_c$ z przesunięciami cyklicznymi.

W projekcie informacja o grafie bazowym jest przechowywana w zestawie tablic ROM inicjalizowanych z plików *.memh:

- **row_ptr** – wskaźniki początków list niezerowych elementów dla kolejnych wierszy (format CSR),
- **col_idx** – indeksy kolumn dla kolejnych niezerowych wpisów,
- **shift** – wartości przesunięć cyklicznych dla par (wiersz, kolumna) zależne od Z_c ,
- **zc_list** – lista dopuszczalnych wartości Z_c (indeksowanie wariantów tablic przesunięć).

Dobór konkretnego zestawu przesunięć dla zadanego Z_c odbywa się poprzez wyszukanie Z_c na liście `zc_list`. Po znalezieniu indeksu Z_c wyznaczany jest adres bazowy do tablicy `shift`, a następnie podczas obliczeń parzystości odczytywane są przesunięcia odpowiadające kolejnym niezerowym elementom grafu.

Takie podejście pozwala obsługiwać wiele wartości Z_c bez przechowywania pełnych macierzy H w pamięci, a jednocześnie zachowuje elastyczność konfiguracji dla BG1/BG2.

Pliki inicjalizacyjne ROM (`*.memh`) nie były przygotowywane ręcznie. Zostały wygenerowane automatycznie przez skrypt MATLAB `gen_ldpc_sparse_roms.m` na podstawie macierzy bazowych (BG1/BG2). Macierze bazowe wykorzystane do budowy tablic ROM są zapisane w katalogu `base_matrices` jako pliki tekstowe o nazwach `NR_<BG>_<Zc>.txt`, tj. osobny plik dla każdej pary (graf bazowy, lifting). Każdy plik zawiera dwuwymiarową macierz liczb całkowitych zapisaną w postaci wierszy tekstu: jeden wiersz macierzy odpowiada jednej linii w pliku. Wartość `-1` oznacza brak połączenia (blok zerowy), natomiast wartości nieujemne z zakresu $0 \dots Z_c - 1$ oznaczają przesunięcie cykliczne bloku permutacyjnego $Z_c \times Z_c$ w danej pozycji macierzy. Rozmiar macierzy jest stały dla danego grafu (BG1: 46×68 , BG2: 42×52), natomiast wartości przesunięć zależą od wybranego Z_c i są zapisane już w postaci odpowiadającej temu liftingowi. Skrypt przekształca macierze bazowe do postaci rzadkiej (listy niezerowych wpisów), a następnie buduje struktury typu CSR (`row_ptr`, `col_idx`) oraz tablicę przesunięć cyklicznych `shift` w wariantach odpowiadających dopuszczalnym wartościom Z_c . Dzięki temu opis grafu w kodzie HDL jest w pełni deterministyczny i spójny ze źródłową definicją macierzy bazowych. Wygenerowane tablice ROM są następnie dołączane do projektu jako pliki inicjalizacji pamięci i odczytywane przez koder w czasie pracy.

4.6.3 Struktura obliczeń: okno Z_c , rotacja i akumulacja XOR

Obliczanie bloków parzystości jest realizowane jako iteracja po wierszach grafu bazowego. Dla każdego wiersza rozpatrywana jest lista niezerowych połączeń (kolumn) oraz odpowiadające im przesunięcia cykliczne. Dla pojedynczego niezerowego wpisu realizowane są trzy kroki:

1. **Wybór fragmentu danych o długości Z_c :** z bufora CB pobierany jest blok danych odpowiadający wskazanej kolumnie (dla części systematycznej) lub blok wyliczony wcześniej (dla części parzystości).
2. **Przesunięcie cykliczne (rotacja):** pobrany blok Z_c jest rotowany o wartość przesunięcia odczytaną z ROM.
3. **Akumulacja XOR:** wynik rotacji jest XOR-owany do akumulatora wiersza.

W implementacji wszystkie rejestry uczestniczące w przetwarzaniu bloków Z_c (bufory danych, bufory po rotacji oraz akumulatory XOR) mają stałą szerokość równą **6 słowom**

64-bit. Wynika to z faktu, że największa obsługiwana wartość liftingu wynosi $Z_c = 384$, co odpowiada $384/64 = 6$ słowom.

Dla bieżącej wartości Z_c logicznie wyznaczana jest liczba aktywnych słów:

$$N = \left\lceil \frac{Z_c}{64} \right\rceil \leq 6$$

i tylko w obrębie tych N słów wykonywane są operacje rotacji oraz akumulacji XOR. Pozostałe słowa (dla $N < 6$) są traktowane jako nieaktywne i nie biorą udziału w obliczeniach. Dodatkowo, jeżeli Z_c nie jest wielokrotnością 64, to w ostatnim aktywnym słowie występują bity nadmiarowe, które są maskowane tak, aby nie wpływały na wynik obliczeń.

Akumulator XOR jest również utrzymywany jako rejestr 6 słów 64-bit. Po przetworzeniu wszystkich niezerowych wpisów danego wiersza, zawartość akumulatora stanowi wynikową wartość odpowiadającego mu bloku parzystości (lub wartości pośredniej wykorzystywanej dalej w procedurze).

4.6.4 Realizacja przesunięcia cyklicznego dla bloku Z_c

Kluczowym problemem implementacyjnym jest efektywna realizacja przesunięcia cyklicznego wektora długości Z_c przy reprezentacji w słowach 64-bit. W projekcie przesunięcie rozdzielono na:

- przesunięcie o wielokrotność 64 bitów (przesunięcie indeksu słowa),
- przesunięcie o pozostałe bity (przesunięcie wewnątrz słowa i sklejenie sąsiednich słów).

Rotacja jest wykonywana poprzez zbudowanie słów wyjściowych z fragmentów dwóch (a w sytuacjach brzegowych trzech) kolejnych słów bufora Z_c . Dodatkowo obsługane są przypadki, w których Z_c nie jest wielokrotnością 64, co wymaga:

- uwzględnienia maskowania bitów nieistotnych w ostatnim słowie,
- poprawnego „zawijania” fragmentów przy rotacji na granicy końca wektora Z_c .

Takie podejście pozwala realizować rotację w sposób deterministyczny i niezależny od konkretnej wartości Z_c , bez konieczności budowania dużych i nieoptymalnych barrel-shifterów dla całego Z_c w jednym bloku.

4.6.5 Wyznaczanie bloków parzystości i obsługa przypadków szczególnych

W kodach QC-LDPC stosowanych w 5G NR pierwsze cztery bloki parzystości $P_0 \dots P_3$ (odpowiadające pierwszym czterem wierszom grafu bazowego) wynikają bezpośrednio ze

struktury *double-diagonal* w macierzy kontrolnej parzystości. Struktura ta gwarantuje, że sposób wyznaczania tych bloków ma stałą, powtarzalną postać: niezależnie od wartości liftingu Z_c . W ramach tego samego grafu bazowego wykonywana jest identyczna sekwencja operacji (pobranie odpowiednich bloków Z_c , rotacje o przesunięcia odczytane z ROM oraz akumulacja XOR). Zmienność Z_c wpływa jedynie na długość przetwarzanych wektorów oraz na konkretne wartości przesunięć cyklicznych, natomiast forma obliczeń pozostaje taka sama.

Dodatkowo w praktyce implementacyjnej istotne jest to, że pomiędzy BG1 i BG2 wspólna część obliczeń obejmuje trzy z czterech pierwszych bloków parzystości: wyznaczanie P_0 , P_1 oraz P_3 przebiega w tej samej strukturze, natomiast różnica pomiędzy grafami ujawnia się jedynie w sposobie wyznaczenia P_2 (trzeciego bloku parzystości). Jest to konsekwencja odmiennego układu połączeń w odpowiadającym mu wierszu grafu, przy zachowaniu tej samej zasady „rotacja + XOR” oraz wykorzystania przesunięć z tablic ROM.

Pierwsze cztery bloki parzystości pełnią w koderze podwójną rolę. Po pierwsze są one częścią końcowego strumienia wyjściowego i muszą zostać zapisane do pamięci bloków parzystości w celu późniejszej emisji. Po drugie stanowią one jedyny zestaw bloków parzystości używany jako punkt odniesienia w obliczaniu kolejnych PB – dalsze wiersze grafu korzystają z zależności opartych o $P_0 \dots P_3$ (wraz z odpowiednio rotowanymi blokami danych). Z tego względu, oprócz zapisu do głównej pamięci PB, wartości $P_0 \dots P_3$ są równolegle utrzymywane w dedykowanych rejestrach roboczych. Umożliwia to ich natychmiastowe wykorzystanie w kolejnych krokach obliczeń bez oczekiwania na synchroniczny odczyt z RAM, co ogranicza liczbę cykli jałowych i upraszcza harmonogram przetwarzania.

4.6.6 Emisja danych: złączenie strumienia CB oraz metadane paddingu

Emisja danych w koderze LDPC jest realizowana jako osobna faza pracy, uruchamiana dopiero po zakończeniu obliczeń dla wszystkich bloków kodowych (CB) należących do przetwarzanego TB. W praktyce oznacza to, że moduł najpierw buforuje i koduje kolejne CB (wyznacza bloki parzystości i zapisuje je do pamięci), a dopiero po zgłoszeniu zakończenia obliczeń (sygnał `pb_done`) przechodzi do sekwencyjnego wypychania wyników na wyjście.

W tej fazie realizowany jest również etap toru LDPC określany jako *złączenie* (*concatenation*) bloków: wcześniej wydzielone CB są na wyjściu sklejjane w **jeden strumień** 64-bit. Kolejność jest stała i powtarzana dla każdego CB:

$$\text{dane CB} \rightarrow \text{bloki parzystości CB} \rightarrow \text{dane następnego CB} \rightarrow \dots$$

co pozwala odbiornikowi traktować wynik jako pojedynczą ramkę wyjściową.

Istotnym elementem emisji jest uwzględnienie załączków mechanizmów związanych z dopasowaniem długości strumienia (rate matching) oraz dziurkowania (puncturing). W aktualnej wersji sprowadza się to do tego, że na wyjściu nie są transmitowane pierwsze $2 \cdot Z_c$ bitów części systematycznej każdego CB. Jest to realizowane poprzez ustawienie początku emisji na `start_bit = 2*zc`, a następnie rozbicie tego przesunięcia na:

- `start_word = start_bit >> 6` — indeks słowa w pamięci CB,
- `bit_off = start_bit[5:0]` — przesunięcie bitowe wewnątrz pierwszego słowa.

W konsekwencji, dla każdego CB emisja danych rozpoczyna się od adresu pierwszego słowa danego CB + `start_word`, a w pierwszym wypuszczanym słowie danego CB wystawiane jest `out_pad_left = bit_off`. Metadana ta informuje, że początek logicznego strumienia znajduje się wewnątrz pierwszego słowa (część MSB należy pominąć).

Dane CB są przechowywane w pamięci jako pełne słowa 64-bit i każdy kolejny CB rozpoczyna się od nowego słowa (wyrównanie adresowe). Dzięki temu, nawet jeśli koniec CB wypada w połowie słowa, nie ma potrzeby „wyciągania” początku następnego CB ze środka tego samego słowa. Niewykorzystane bity na końcu CB są traktowane jako dopełnienie po stronie LSB, a ich liczba jest sygnalizowana metadanymi `out_pad_right`. W implementacji dla zakończenia części systematycznej CB wartość ta jest ustawiana na podstawie `data_bits[5:0]` (czyli reszty z dzielenia długości danych przez 64), zgodnie z zależnością `out_pad_right = 64 - data_bits[5:0]` na ostatnim słowie danych CB.

Analogicznie zorganizowana jest emisja bloków parzystości. Każdy blok parzystości ma długość Z_c bitów, ale wewnętrznie jest reprezentowany w postaci maksymalnie 6 słów 64-bit, ponieważ maksymalny lifting w 5G NR wynosi $Z_c = 384$, a więc $\lceil 384/64 \rceil = 6$. Dla danej konfiguracji do pamięci RAM zostały zapisane tylko aktywne słowa. Jeżeli Z_c nie jest wielokrotnością 64, w ostatnim słowie bloku występują bity nadmiarowe po stronie LSB – są one traktowane jako nieistotne, a `out_pad_right` jest ustawiane na $64 - (Z_c \bmod 64)$ dla ostatniego słowa danego bloku parzystości. Co ważne, również bloki parzystości są w pamięci ułożone tak, aby każdy blok zaczynał się na granicy słowa 64-bit, dzięki czemu podczas emisji nie występuje przypadek, w którym kolejny blok parzystości rozpoczyna się „w połowie” słowa.

Sygnał `out_last` jest generowany tylko raz – na końcu emisji bloków parzystości ostatniego CB. Oznacza to, że całe TB jest traktowane jako jedna ramka wyjściowa, a granice CB są implicitne (wynikają z metadanych i znanych długości), natomiast formalnym zakończeniem strumienia jest dopiero zakończenie ostatniego CB.

Przyjęta organizacja emisji (wyrównywanie CB i PB do słów 64-bit oraz brak rozpoczynania kolejnych fragmentów w środku słowa) upraszcza sterowanie i adresowanie pamięci, ale jest rozwiązaniem nieoptymalnym pamięciowo. Pojawiają się „puste” bity w słowach brzegowych oraz dodatkowe narzuty wynikające z wyrównań do 64 bit. Jest to

świadomy kompromis i naturalny punkt wyjścia do dalszego rozwoju: możliwe jest zagęszczenie buforów oraz bardziej zwarta emisja bez sztucznych granic słowa pomiędzy fragmentami, kosztem bardziej złożonej logiki składania danych na wyjściu.

4.6.7 Załączki puncturingu i rate matchingu

W standardowym torze 5G NR po kodowaniu LDPC występuje etap rate matchingu, który m.in. usuwa (nie transmituje) wybrane bity oraz mapuje wynik do żądanej długości E w zależności od MCS i redundancji. W ramach niniejszego projektu nie zaimplementowano pełnego rate matchingu (circular buffer, dobór E , RV itp.), jednak w koderze zawarto dwa elementy będące bezpośrednim punktem wyjścia do jego rozbudowy:

- **Filler bits:** bity dopełniające nie są transmitowane na wyjściu. Są one uwzględniane logicznie w wyznaczaniu długości i granic strumienia (m.in. przez `in_filler_cnt`), ale nie pojawiają się jako dane użytkowe w emitowanym strumieniu.
- **Pominięcie dwóch pierwszych bloków Z_c :** na wyjściu część systematyczna jest emitowana z przesunięciem o dwa pierwsze bloki długości Z_c , tzn. strumień zaczyna się od bitu po tych dwóch blokach. Jest to zgodne z referencyjnym modelem MATLAB użytym do weryfikacji i odpowiada mechanizmowi puncturingu stosowanemu w 5G NR dla kodów QC-LDPC.

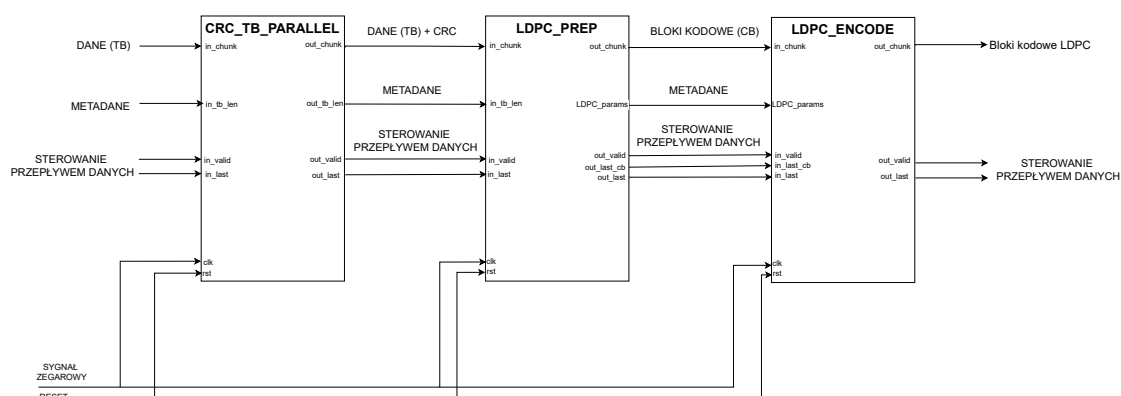
W obecnej wersji kodera powyższe mechanizmy należy traktować jako przygotowanie pod pełną implementację kanału PDSCH systemu 5G NR.

4.7 Integracja top-level i sterowanie pracą toru

Moduł PDSCHTx stanowi warstwę integrującą cały tor przetwarzania, łącząc kolejne etapy zgodnie z architekturą przedstawioną na rys. 4.5. W ujęciu funkcjonalnym łańcuch przetwarzania ma postać:

TB (64b stream) $\rightarrow$ CRC_TB_PARALLEL $\rightarrow$ LDPC_PREP $\rightarrow$ LDPC_ENCODE $\rightarrow$ strumień
zakodowany

Łączenie bloków i przepływ danych. Wejściowy strumień TB (`in_chunk`, `in_valid`, `in_last`, `in_tb_len`) jest w pierwszym kroku kierowany do modułu CRC_TB_PARALLEL, który dopisuje CRC TB i przekazuje dane dalej w tym samym formacie strumieniowym. Następnie LDPC_PREP wykonuje operacje przygotowawcze do kodowania LDPC (m.in. segmentację TB na CB oraz dopisanie CRC CB) i generuje metadane sterujące kodowaniem (`LDPC_params`).

PDSCH

Rysunek 4.5: Integracja toru w module PDSCHTx.

Moduł LDPC_ENCODE przyjmuje strumień bloków kodowych (CB) oraz odpowiadające im metadane i realizuje kodowanie. Na wyjściu koder powstaje pojedynczy strumień danych odpowiadający całemu TB po kodowaniu LDPC (z zachowaniem przyjętego w projekcie sposobu emisji danych).

Sterowanie granicami TB i CB. W top-level zastosowano rozdzielenie sygnałów końca ramki na dwa poziomy:

- **Koniec TB** (*in_last* / *out_last*) – występuje tylko raz dla całego bloku transportowego i zamyka przetwarzanie całej ramki w torze.
- **Koniec CB** (*out_last_cb*) – sygnał wewnętrzny pomiędzy LDPC_PREP i LDPC_ENCODE, służący do jednoznacznego oznaczania granicy kolejnych bloków kodowych w strumieniu. Pozwala to koderowi poprawnie rozdzielać CB bez potrzeby dodatkowego „ręcznego” liczenia słów na wejściu.

Takie podejście upraszcza integrację: tor może traktować TB jako jedną ramkę wejściową, natomiast koder LDPC dostaje dodatkową informację o końcu aktualnego CB, co jest naturalne w kontekście segmentacji i późniejszej rekombinacji danych.

Na poziomie PDSCHTx integracja sprowadza się więc do poprawnego połączenia sygnałów *valid/last* pomiędzy blokami oraz zapewnienia, że granice TB/CB są zachowane i

jednoznaczne na kolejnych etapach przetwarzania.

Reset i cykl pracy toru. Reset (**rst**) jest wspólny dla całego toru i inicjalizuje wszystkie etapy przetwarzania do stanu spoczynkowego. Zgodnie z przyjętymi założeniami projektowymi tor obsługuje pojedynczy TB w jednym cyklu pracy, a rozpoczęcie przetwarzania następnego TB wymaga ponownego resetu.

Rozdział 5

Weryfikacja i walidacja

5.1 Cele i kryteria poprawności

Celem weryfikacji i walidacji jest potwierdzenie, że zaimplementowany tor przetwarzania (od wejściowego strumienia TB do wyjściowego strumienia po kodowaniu LDPC) działa zgodnie z przyjętą specyfikacją, jest deterministyczny oraz generuje wynik bitowo zgodny z danymi referencyjnymi dla zestawu przypadków testowych obejmujących różne konfiguracje długości TB i parametrów kodowania. Weryfikacja jest realizowana przede wszystkim w symulacji funkcjonalnej z wykorzystaniem wektorów referencyjnych generowanych w MATLAB, co umożliwia bezpośrednie porównanie wyników symulacji HDL z danymi idealnymi.

5.1.1 Cele weryfikacji

W ramach pracy przyjęto następujące cele weryfikacji:

- **Poprawność funkcjonalna toru:** potwierdzenie poprawnego przetwarzania TB w pełnym łańcuchu (CRC dla TB, przygotowanie do LDPC, kodowanie LDPC i emisja wyniku) przy zachowaniu stałej szerokości słowa 64-bit.
- **Deterministyczność:** dla identycznych danych wejściowych oraz identycznych metadanych sterujących tor powinien generować identyczny strumień wyjściowy.
- **Bitowo-dokładna zgodność z referencją:** weryfikacja poprzez porównanie strumienia wyjściowego z wektorami referencyjnymi w sposób bitowo-dokładny.

5.1.2 Kryteria poprawności

Za poprawne uznaje się działanie, które spełnia jednocześnie poniższe kryteria:

- **Zgodność danych:** bity danych wyjściowych są zgodne z danymi referencyjnymi dla danego przypadku testowego, przy uwzględnieniu przyjętej kolejności bitów MSB-first.
- **Zgodność ramek i sygnałów sterujących:** semantyka sygnałów `valid/last` jest zachowana: `valid` oznacza ważność słowa w danym cyklu, natomiast `out_last` występuje tylko raz i zamyka emisję całego TB na wyjściu toru.
- **Poprawna interpretacja bitów brzegowych:** w przypadkach, w których granice logiczne wypadają wewnątrz słowa 64-bit, strumień musi zawierać jednoznaczne metadane pozwalające odtworzyć dokładny zakres bitów danych. W projekcie rolę tę pełnią sygnały `out_pad_left` oraz `out_pad_right`, które informują, ile bitów należy pominąć odpowiednio od strony MSB oraz LSB.
- **Odporność na przerwy w strumieniu wejściowym:** dopuszczalne są cykle z `in_valid=0` pomiędzy kolejnymi słowami TB; słowo jest próbkowane wyłącznie w cyklu, w którym `in_valid=1`.
- **Spójność zachowania względem przyjętego modelu pracy:** przetwarzanie obejmuje pojedynczy TB pomiędzy resetami, a rozpoczęcie kolejnego przypadku testowego wymaga resetu, co zapewnia jednoznaczne odseparowanie transakcji.

5.2 Środowisko weryfikacji

Weryfikację poprawności działania toru przeprowadzono w oparciu o symulację funkcjonalną modułów HDL oraz porównanie wyników z danymi referencyjnymi (tzw. *golden vectors*). Środowisko weryfikacji składa się z trzech głównych elementów: symulatora HDL, zestawu testbenchy oraz środowiska MATLAB użytego do generacji i analizy wektorów testowych.

5.2.1 Symulacja HDL (Questa)

Symulacje funkcjonalne wykonano z użyciem symulatora Questa (Intel FPGA Edition). Narzędzie to zostało wykorzystane do:

- kompilacji i uruchamiania testbenchy napisanych w SystemVerilog,
- analizy przebiegów czasowych (waveforms) oraz stanów wewnętrznych w procesie debugowania,
- wykonywania automatycznego porównania wyników DUT (Device Under Test) z danymi referencyjnymi.

Weryfikacja koncentruje się na poprawności funkcjonalnej (bitowo-dokładnej), a nie na dokładnych opóźnieniach czasowych w sensie fizycznej implementacji.

5.2.2 Testbenche i automatyzacja przebiegu testów

Dla poszczególnych modułów opracowano dedykowane testbenche według wspólnego szkieletu:

- wczytanie wektorów wejściowych i metadanych z plików tekstowych,
- podanie strumienia do DUT z zachowaniem sygnałów `valid/last` i kolejności bitów MSB-first,
- zapis strumienia wygenerowanego przez DUT do pliku wynikowego,
- porównanie danych wyjściowych z wektorem referencyjnym w sposób bitowo-dokładny,
- generowanie raportu testu (informacje o niezgodnościach, pozycjach błędów itp.).

Jednolita struktura testbenchy ułatwia dodawanie nowych przypadków testowych oraz rozszerzanie pokrycia o kolejne konfiguracje parametrów.

5.2.3 Wektory referencyjne i narzędzia MATLAB

Wektory testowe oraz dane referencyjne generowane są w MATLAB, który był używany także jako narzędzie pomocnicze do:

- weryfikacji oczekiwanego formatu strumienia (np. kolejności bitów, obsługi paddingu),
- wizualizacji pośrednich etapów przetwarzania,
- analizy różnic w przypadku wykrycia rozbieżności pomiędzy DUT a referencją.

5.3 Organizacja danych testowych i wektorów referencyjnych

Weryfikacja projektu opiera się na porównaniu wyników symulacji z danymi referencyjnymi (*golden vectors*) wygenerowanymi w środowisku MATLAB. Dane testowe są przechowywane w uporządkowanej strukturze katalogów, w której każdy test identyfikowany jest unikalną nazwą przypadku (np. A1477, A12107 itd.). Nazwa przypadku jest wspólnym prefiksem dla plików wejściowych, metadanych oraz oczekiwanych wyników.

5.3.1 Struktura katalogów

Zestawy wektorów zebrane są w katalogu `vectors/` i pogrupowane według weryfikowanego bloku funkcjonalnego toru. Każdy moduł posiada podkatalog `Output_Matlab` zawierający dane referencyjne oraz `Output_DUT` gdzie zaspisywane są wyniki wygenerowane przez DUT:

- `vectors/TB_crc/` – wektory dla CRC dopisywanego do TB (warianty szeregowy i równoległy),
- `vectors/CB_crc/` – wektory dla CRC dopisywanego do CB (CRC24B),
- `vectors/LDPC_prep/` – wektory dla przygotowania danych do kodowania LDPC,
- `vectors/LDPC_encode/` – wektory dla kodera LDPC,
- `vectors/PDSCH/` – wektory dla modułu integrującego cały tor PDSCH,
- `vectors/Parallel_Matrices/` – pliki zawierające wygenerowane równania XOR dla modułu `crc_update` (warianty CRC16, CRC24A, CRC24B); są to gotowe przypisania postaci `crc_out[i] = ...` zależne od `crc_in[]` oraz `din[]`.

5.3.2 Nazewnictwo plików i typy danych

Dla danego przypadku testowego `Axxxx` wykorzystywane są zazwyczaj trzy podstawowe pliki w `Output_Matlab`:

- `Axxxx_in_bits.txt` – strumień wejściowy w postaci sekwencji bitów,
- `Axxxx_out_bits.txt` – oczekiwany strumień wyjściowy w postaci sekwencji bitów,
- `Axxxx_meta.txt` – metadane opisujące parametry przypadku testowego.

Dla modułów CRC dodatkowo występuje plik:

- `Axxxx_crc_only.txt` – oczekiwane bity CRC (bez danych użytkownika).

W katalogach `Output_DUT` testbench zapisuje wyniki DUT w plikach o nazwach z sufiksem `_dut_...`, np.:

- `Axxxx_dut_crc_out.txt`,
- `Axxxx_dut_stream_out.txt`,
- `Axxxx_dut_out_bits.txt`,
- `Axxxx_dut_out_cb.txt`.

Dokładny sufix zależy od modułu i sposobu zapisu wyjścia w danym teście.

5.3.3 Format plików bitowych

Pliki `Axxxx_in_bits.txt` oraz `Axxxx_out_bits.txt` przechowują strumień jako znaki 0 i 1, gdzie `Axxxx` jest identyfikatorem przypadku testowego. W zależności od modułu strumień może być zapisany:

- jako jedna długa linia bitów (ciąg znaków),
- jako kilka linii (np. podział na fragmenty lub bloki),
- z separatorami pomiędzy blokami kodowymi.

W testach związanych z segmentacją oraz kodowaniem LDPC w plikach generowanych w MATLAB mogą pojawiać się znaczniki `-1`, które reprezentują bity typu *filler* (dopełnienie logiczne w części systematycznej CB). Nie są to dane binarne (0/1) przeznaczone do transmisji, dlatego na etapie porównywania wyników testbench pomija wszystkie wystąpienia `-1`. Porównanie z wyjściem DUT wykonywane jest wyłącznie dla rzeczywistych bitów strumienia, tj. 0 i 1.

5.3.4 Format plików metadanych

Pliki `Axxxx_meta.txt` mają postać tekstową `klucz=wartość`, po jednej parze na linię. Zawierają parametry niezbędne do skonfigurowania testu oraz do interpretacji strumienia. Zakres metadanych zależy od modułu. W praktyce spotykane są m.in.:

- identyfikator przypadku: `NAME`,
- długości bitowe na wybranych etapach toru (np. `A`, `B`, `BC`, `K`, `KCB`),
- wybór grafu bazowego i liftingu (np. `BG`, `ZC`),
- informacja o dopełnieniach (`FILLER_CNT`),
- parametry CRC (np. `POLY`, `L`).

Metadane stanowią punkt wspólny pomiędzy MATLAB a testbenchami: MATLAB zapisuje parametry testu, a testbench odczytuje je i podaje na wejścia modułu w HDL. Służą również do ułatwienia analizy plików wyjściowych przyspieszając oddzielenie poszczególnych bloków strumienia.

5.4 Testy modułowe

Testy modułowe mają na celu potwierdzenie poprawności funkcjonalnej poszczególnych bloków toru. Weryfikacja opiera się na porównaniu wyników symulacji (DUT) z

wektorami referencyjnymi generowanymi w MATLAB. Testy są realizowane w symulatorze Questa z użyciem dedykowanych testbenchy w języku SystemVerilog, zbudowanych według wspólnego szkieletu.

5.4.1 Dane testowe, organizacja przypadków i format plików

Każdy przypadek testowy jest identyfikowany parametrem CASE w postaci **Axxxx** (np. A1477, A12107). Dla każdego modułu zestaw danych jest uporządkowany w analogiczny sposób: źródłem referencji są pliki generowane w MATLAB, a testbench zapisuje wyniki DUT do osobnego katalogu wynikowego.

Typowy komplet plików dla przypadku CASE obejmuje:

- `<CASE>_in_bits.txt` – wejściowy strumień bitów (0/1),
- `<CASE>_out_bits.txt` – referencyjny strumień wyjściowy (0/1),
- `<CASE>_meta.txt` – metadane w formacie **KEY=VALUE**.

W plikach `meta` występują parametry opisujące przypadek (np. długości na poszczególnych etapach, wybór grafu bazowego i liftingu, informacje o dopełnieniach). Dla modułu top-level PDSCHTx testbench wykorzystuje wprost długość TB zapisaną w metadanych i ustawia `tb_len`.

W ramach pracy użyto stałego zestawu **20** przypadków testowych (wspólnego dla weryfikowanych bloków): {A57, A463, A983, A1477, A2193, A3187, A4219, A5297, A6473, A7991, A9839, A12107, A14963, A18371, A22159, A26841, A31777, A38693, A46927, A59761}.

- **Graf bazowy:** BG1 – 14 przypadków, BG2 – 6 przypadków.
- **Liczba bloków kodowych:** $CB_COUNT \in \{1, 2, 3, 4, 5, 6, 8\}$, w tym: 1 CB – 10 przypadków, 2 CB – 3 przypadki, 3 CB – 2 przypadki, 4 CB – 2 przypadki, 5 CB – 1 przypadek, 6 CB – 1 przypadek, 8 CB – 1 przypadek.
- **Lifting:** $Z_c \in \{13, 60, 104, 160, 208, 224, 240, 256, 288, 320, 352, 384\}$, przy czym w zestawie: $Z_c = 352$ występuje 4 razy, $Z_c = 384$ występuje 4 razy, $Z_c = 288$ i $Z_c = 320$ po 2 razy, pozostałe wartości po 1 razie.
- **Zakres długości TB:** A od 57 do 59761 bitów.
- **Ogon TB względem 64 bitów:** w tym zestawie wszystkie przypadki spełniają $A \bmod 64 \neq 0$, czyli każdy test wymusza obsługę niepełnego ostatniego słowa TB.

5.4.2 Wspólny szkielet testbenchy i zasady porównywania

Każdy testbench:

- wybiera przypadek testowy poprzez parametr **CASE**,
- wczytuje pliki wejściowe, referencyjne oraz metadane,
- rekonstruuje strumień wejściowy w kolejności **MSB-first**,
- podaje dane do DUT z użyciem sterowania **valid/last**,
- zapisuje strumień wygenerowany przez DUT do pliku w katalogu wynikowym,
- wykonuje porównanie bitowo-dokładne z referencją oraz generuje raport diagnostyczny.

Istotną cechą testbenchy jest to, że nie przerywają symulacji przy pierwszej niezgodności: błędy są zliczane i raportowane, a test kontynuuje pracę aż do warunku zakończenia (zwykle **out_last**) lub do osiągnięcia limitu czasu (**TIMEOUT**). Takie podejście ułatwia diagnostykę, ponieważ pozwala zobaczyć pełny „kształt” rozjazdu (np. przesunięcie, utrata fragmentu, dodatkowe bity), a nie tylko pierwszy błąd.

W testach związanych z LDPC w plikach referencyjnych mogą występować tokeny **-1**. W projekcie odpowiadają one bitom typu *filler* i są pomijane w porównaniu (porównanie dotyczy wyłącznie bitów 0/1), ponieważ DUT nie emituje fillerów jako bitów binarnych na wyjściu.

5.4.3 Test modułu **crc_serial**

Test modułowy dla **crc_serial** weryfikuje szeregowe (1 bit/cykl) dopisywanie CRC:

- wejściem jest strumień bitów 0/1 z **in_valid** oraz **in_last**,
- wyjściem jest strumień wiadomość + CRC, gdzie **out_last** występuje na ostatnim bicie CRC,
- testbench zapisuje strumień DUT do pliku oraz porównuje go bitowo z referencją.

Dane wyjściowe są zapisywane w postaci strumienia bitów w jednej linii.

5.4.4 Test modułów **crc_tb_parallel** oraz **crc_cb_parallel**

Testy **crc_tb_parallel** oraz **crc_cb_parallel** weryfikują dopisywanie CRC w torze 64-bitowym (z uwzględnieniem końcówki TB/CB, która może być niepełnym słowem). Testbench podaje kolejne słowa 64-bit z **in_valid/in_last**, konfiguruje DUT metadanymi, porównuje strumień wyjściowy z referencją oraz zapisuje wynik DUT do pliku. Dane wyjściowe są zapisywane w postaci strumienia bitów w jednej linii.

5.4.5 Test modułu LDPC_prep

Testy LDPC_prep obejmują poprawność segmentacji TB na CB, dopisania CRC24B dla CB, przygotowania strumienia danych CB w formacie wejściowym dla kodera LDPC oraz generacji metadanych sterujących. W przypadku obecności tokenów -1 w plikach referencyjnych są one ignorowane.

Ponieważ długość danych w CB nie musi być wielokrotnością 64, w ostatnim słowie danego CB mogą pojawić się bity dopełnienia (zera) wynikające wyłącznie z wyrównania do granicy słowa 64-bit. Na etapie weryfikacji porównywane są wyłącznie bity faktycznie należące do CB (zgodnie z długością/metadanymi), natomiast bity dopełnienia nie wpływają na wynik porównania.

Dane wyjściowe są zapisywane w postaci słowa 64-bitowego na linię (MSB-first). Po odebraniu sygnału out_last wpisywanych jest kilka znaków nowej linii w celu wizualnego oddzielenia CB.

5.4.6 Test modułu LDPC_encode

Testy LDPC_encode obejmują poprawność kodowania QC-LDPC dla różnych konfiguracji (BG, Z_c , liczba filler bits), zgodność bitowo-dokładną z referencją MATLAB oraz poprawną obsługę ramek i zakończenia całego TB.

Ponieważ strumień wyjściowy jest przenoszony w słowach 64-bit, DUT wystawia metadane out_pad_left oraz out_pad_right opisujące liczbę bitów nieistotnych w danym słowie odpowiednio od strony MSB i LSB. Zapis do pliku oraz porównanie z referencją wykonywane są wyłącznie dla bitów po uwzględnieniu paddingu (tj. po odrzuceniu out_pad_left bitów z początku słowa i out_pad_right bitów z końca słowa). Dzięki temu bity dopełnienia wynikające z wyrównań do 64 bitów nie wpływają na wynik weryfikacji.

Dane wyjściowe są zapisywane w postaci słowa 64-bitowego na linię (MSB-first) po zastosowaniu selekcji bitów.

5.4.7 Test integracyjny modułu top-level PDSCHTx

Dla modułu integrującego tor (PDSCHTx) zastosowano testbench utrzymany w tej samej konwencji, co testy modułowe: wejście + golden + meta z MATLAB oraz porównanie bitowo-dokładne wyjścia DUT. Test weryfikuje poprawność współdziałania modułów w jednym strumieniu danych, a zakończenie całej transakcji jest sygnalizowane pojedynczym impulsem out_last na końcu TB.

Dane wyjściowe są zapisywane i porównywane identycznie jak w przypadku LDPC_encode, tj. z uwzględnieniem out_pad_left/out_pad_right oraz pominięciem tokenów -1 w referencji.

5.5 Analiza wyników testów i symulacji

Wszystkie przypadki testowe zakończyły się poprawnie (brak rozbieżności bitowych względem referencji MATLAB), dlatego analiza skupia się na metrykach czasowych uzyskanych z symulacji. Dla każdego modułu oraz wybranych przypadków testowych zebrano czasy (w cyklach zegara) odpowiadające kluczowym zdarzeniom na interfejsie `valid/last`.

5.5.1 Definicje zdarzeń i metryk czasowych

Dla każdego przypadku testowego rejestrowane są cztery znaczniki czasowe (w cyklach zegara):

- $t_{\text{in_first}}$ – pierwszy cykl z poprawnym wejściem (`in_valid=1`),
- $t_{\text{in_last}}$ – cykl ostatniego elementu wejścia (`in_valid=1 && in_last=1`),
- $t_{\text{out_first}}$ – pierwszy cykl z poprawnym wyjściem (`out_valid=1`),
- $t_{\text{out_last}}$ – cykl zakończenia ramki (`out_valid=1 && out_last=1`).

Na tej podstawie wyznaczono następujące metryki:

$$L_{\text{start}} = t_{\text{out_first}} - t_{\text{in_first}}, \quad T_{\text{in}} = t_{\text{in_last}} - t_{\text{in_first}} + 1,$$

$$T_{\text{flush}} = t_{\text{out_last}} - t_{\text{in_last}}, \quad T_{\text{total}} = t_{\text{out_last}} - t_{\text{in_first}} + 1.$$

Metryka L_{start} opisuje opóźnienie pojawienia się pierwszego wyniku, T_{in} czas przyjmowania danych wejściowych, T_{flush} czas „domknięcia” po zakończeniu wejścia (np. dopisanie CRC, kodowanie, emisja), natomiast T_{total} całkowity czas przetworzenia ramki liczony od pierwszego elementu wejścia do `out_last`.

5.5.2 Wybrane przypadki testowe i ich parametry

Do analizy wybrano 7 przypadków testowych, obejmujących skrajnie krótkie TB, przypadki w pobliżu progu doboru CRC, oraz przypadki wieloblokowe (większa liczba CB). Parametry pochodzą z plików `<CASE>_meta.txt` wygenerowanych w MATLAB.

Zestawienie parametrów analizowanych przypadków znajduje się w tabeli 5.1.

5.5.3 Latencja modułu `crc_serial`

Tabela 5.2 przedstawia metryki czasowe modułu `crc_serial`. W tym wariantcie dane są przetwarzane bit-po-bicie (1 bit/cykl), dlatego T_{in} jest wprost równe długości TB w bitach (**A**). Stała różnica pomiędzy $t_{\text{in_last}}$ i $t_{\text{out_last}}$ wynika z dopisania CRC po zakończeniu danych (ogon ramki), co znajduje odzwierciedlenie w T_{flush} .

Tabela 5.1: Parametry wybranych przypadków testowych (metadane MATLAB).

CASE	A [b]	CRC(TB)	BG	Z_c	C	N_{filler}
A57	57	CRC16	2	13	1	57
A3187	3187	CRC16	2	352	1	317
A4219	4219	CRC24A	1	208	1	333
A12107	12107	CRC24A	1	288	2	246
A18371	18371	CRC24A	1	288	3	180
A26841	26841	CRC24A	1	320	4	299
A59761	59761	CRC24A	1	352	8	246

Tabela 5.2: Metryki czasowe modułu `crc_serial` (cykle).

CASE	L_{start}	T_{in}	T_{flush}	T_{total}
A57	1	57	17	74
A3187	1	3187	17	3204
A4219	1	4219	25	4244
A12107	1	12107	25	12132
A18371	1	18371	25	18396
A26841	1	26841	25	26866
A59761	1	59761	25	59786

5.5.4 Latencja modułu `crc_tb_parallel`

W analizie pominięto osobne testy `crc_cb_parallel`, ponieważ architektura oraz sposób działania są równoważne z `crc_tb_parallel`.

Wyniki dla `crc_tb_parallel` zestawiono w tabeli 5.3. W porównaniu do wariantu szeregowego, przetwarzanie pełnych słów 64-bit odbywa się równolegle, a operacje bitowe są ograniczone do końcówki TB (niepełnego słowa) oraz dopisania CRC. Metryka T_{flush} pozwala oszacować koszt „domknięcia” ramki po przyjęciu ostatniego słowa wejściowego.

Tabela 5.3: Metryki czasowe modułu `crc_tb_parallel` (cykle).

CASE	L_{start}	T_{in}	T_{flush}	T_{total}
A57	68	1	77	78
A3187	1	50	71	121
A4219	1	66	87	153
A12107	1	190	39	229
A18371	1	288	31	319
A26841	1	420	53	473
A59761	1	934	77	1011

5.5.5 Latencja modułu `LDPC_prep`

Tabela 5.4 pokazuje metryki czasowe modułu `LDPC_prep`. W tym etapie znaczną część czasu stanowi przygotowanie CB (segmentacja, dopisanie CRC24B, wyznaczenie paramet-

trów), co ujawnia się w dużym L_{start} oraz T_{flush} dla dłuższych przypadków. Ponieważ moduł generuje strumień CB oraz metadane sterujące, czas przetwarzania rośnie wraz z długością TB i liczbą CB.

Tabela 5.4: Metryki czasowe modułu LDPC_prep (cykle).

CASE	L_{start}	T_{in}	T_{flush}	T_{total}
A57	21	2	21	23
A3187	329	51	329	380
A4219	201	67	201	268
A12107	6388	190	6734	6924
A18371	6458	288	6988	7276
A26841	7071	420	7851	8271
A59761	7866	935	9439	10374

5.5.6 Latencja modułu LDPC_encode

Wyniki latencji kodera LDPC_encode przedstawiono w tabeli 5.5. Zgodnie z architekturą projektu koder przechodzi do fazy emisji dopiero po zakończeniu przetwarzania wszystkich CB, co powoduje, że T_{flush} dominuje w całkowitym czasie T_{total} . Jest to kluczowy element wpływający na opóźnienie całego toru.

Tabela 5.5: Metryki czasowe modułu LDPC_encode (cykle).

CASE	L_{start}	T_{in}	T_{flush}	T_{total}
A57	1784	2	1826	1828
A3187	4612	51	4852	4903
A4219	6583	67	6760	6827
A12107	14856	96	15395	15491
A18371	22262	97	23122	23219
A26841	30363	106	31566	31672
A59761	70180	118	73139	73257

5.5.7 Latencja toru top-level PDSCHTx

Tabela 5.6 zawiera metryki czasowe modułu top-level PDSCHTx. Wartości te obejmują łączne opóźnienie toru przetwarzania od przyjęcia TB do zakończenia emisji. Porównanie tabeli 5.6 z tabelą 5.5 pokazuje, że latencja całego toru jest w największym stopniu determinowana przez etap kodowania LDPC.

5.5.8 Wnioski z analizy latencji

Dla wszystkich analizowanych przypadków dominującym składnikiem czasu przetwarzania jest moduł LDPC_encode. Wynika to z faktu, że w tej architekturze koder wykonuje

Tabela 5.6: Metryki czasowe modułu top-level PDSCHTx (cykle).

CASE	L_{start}	T_{in}	T_{flush}	T_{total}
A57	1873	1	1915	1916
A3187	4942	50	5273	5323
A4219	6785	66	6963	7029
A12107	21588	190	22033	22223
A18371	29246	288	29915	30203
A26841	38208	420	39097	39517
A59761	79604	934	81747	82681

obliczenia po zakończeniu przyjęcia danych i przechodzi do fazy emisji dopiero po przetworzeniu pełnego zestawu CB, co powoduje, że T_{flush} stanowi większość czasu T_{total} .

Moduły CRC (zarówno wariant szeregowy, jak i równoległy) mają relatywnie niewielki wpływ na całkowitą latencję toru, a ich czas domknięcia jest ograniczony głównie do dopisania CRC oraz obsługi niepełnej końcówki.

5.5.9 Narzut magistrali 64-bit

Ponieważ tor pracuje na stałej magistrali 64-bit, nie w każdym cyklu wszystkie bity słowa niosą dane użyteczne. Granice logicznych danych nie muszą pokrywać się z granicami słów, dlatego moduł PDSCHTx wystawia metadane `out_pad_left` oraz `out_pad_right`, które określają liczbę bitów do pominięcia odpowiednio od strony MSB i LSB. Dla j -tego słowa wyjściowego liczba bitów użytecznych wynosi:

$$v_j = 64 - \text{out_pad_left}_j - \text{out_pad_right}_j.$$

Sumaryczna liczba bitów użytecznych w strumieniu to $V = \sum_j v_j$, natomiast liczba bitów „przeniesionych” magistralą (licząc 64 bity na każde słowo z `out_valid=1`) wynosi $64 \cdot N$, gdzie N to liczba słów wyjściowych. Z tego wyznacza się narzut oraz wykorzystanie magistrali:

$$B_{\text{waste}} = 64 \cdot N - V, \quad \eta = \frac{V}{64 \cdot N}.$$

W testbenchach dla PDSCHTx strumień DUT jest zapisywany do pliku wynikowego już po uwzględnieniu `out_pad_left/right`, tzn. do pliku trafiają wyłącznie bity uznane za ważne w danym słowie. W praktyce oznacza to, że:

- N odpowiada liczbie niepustych linii w pliku `<CASE>_dut_out_bits.txt`,
- V jest sumą długości tych linii (liczby zapisanych bitów 0/1),
- $\bar{v} = V/N$ opisuje średnią liczbę bitów użytecznych na jedno słowo magistrali.

Wyniki dla analizowanych przypadków przedstawiono w tabeli 5.7. Zauważalne jest, że

dla bardzo krótkich ramek (np. A57) narzut wyrównań do 64 bitów dominuje, natomiast dla dłuższych ramek wykorzystanie magistrali rośnie i zbliża się do 100%.

Tabela 5.7: Narzut wynikający z pracy na magistrali.

CASE	N [słów]	V [bitów]	η [%]	B_{waste} [bitów]	$\bar{v}$ [bit/słowo]
A57	44	593	21.06	2223	13.48
A3187	292	17283	92.48	1405	59.19
A4219	245	13395	85.43	2285	54.67
A12107	634	37524	92.48	3052	59.19
A18371	954	56484	92.51	4572	59.21
A26841	1304	83284	99.79	172	63.87
A59761	3064	183888	93.77	12208	60.02

5.6 Wyniki syntezy i analiza implementacyjna w FPGA

W niniejszej sekcji przedstawiono wyniki pełnej kompilacji projektu w środowisku Intel Quartus Prime (Analiza i Synteza, Fitter oraz analiza czasowa TimeQuest). Celem jest ocena realizowalności projektu w wybranym układzie FPGA oraz wskazanie ograniczeń czasowych wynikających z bieżącej implementacji.

5.6.1 Konfiguracja narzędzi i założenia kompilacji

Tabela 5.8 podsumowuje konfigurację narzędzi oraz docelowego układu FPGA. Analiza czasowa została wykonana w TimeQuest na podstawie pliku ograniczeń SDC, w którym zdefiniowano pojedynczy zegar `clk` o okresie 10 ns (100 MHz).

Tabela 5.8: Konfiguracja kompilacji i docelowego układu FPGA.

Element	Wartość
Narzędzie	Intel Quartus Prime Lite 25.1std.0 (Build 1129, 2025-10-21)
Top-level entity	PDSCHTx
Rodzina FPGA	Cyclone V
Układ docelowy	5CSXFC6D6F31C6
Plik SDC	<code>rtl/clk.sdc</code> (Status: OK)
Zegar	<code>clk</code> , okres 10.000 ns (100.0 MHz)

5.6.2 Wykorzystanie zasobów układu

Zestawienie wykorzystania zasobów po etapie Fitter przedstawiono w tabeli 5.9. Wyniki potwierdzają, że projekt mieści się w docelowym układzie FPGA z istotnym zapasem zasobów, szczególnie w obszarze pamięci blokowej oraz jednostek DSP.

Tabela 5.9: Wykorzystanie zasobów po kompilacji (Fitter Summary).

Zasób	Wykorzystanie	Dostępne	Udział
ALM	12 177	41 910	29%
Rejestry	6 805	–	–
Piny I/O	162	499	32%
Pamięć (bity)	393 216	5 662 720	7%
Bloki RAM	46	553	8%
DSP	5	112	4%
PLL	0	15	0%

5.6.3 Analiza czasowa i maksymalna częstotliwość pracy

Analiza czasowa została przeprowadzona w TimeQuest dla zegara `clk` o wymaganym okresie 10 ns (100 MHz). Podsumowanie wyników dla najważniejszych sprawdzeń (setup/hold oraz minimalna szerokość impulsu) pokazano w tabeli 5.10.

Weryfikacja *setup* nie została spełniona dla najgorszego narożnika (Slow 1100mV 85°C), co objawia się ujemnym slackiem. Oznacza to, że przy zadanym ograniczeniu 100 MHz projekt wymaga optymalizacji (logiki lub architektury) albo obniżenia częstotliwości zegara. Z drugiej strony sprawdzenia *hold* oraz minimalnej szerokości impulsu (pulse width) są spełnione (slack dodatni).

Tabela 5.10: Podsumowanie analizy czasowej (TimeQuest).

Typ sprawdzenia	Corner	Worst Slack [ns]	Wniosek
Setup (wymóg 10 ns)	Slow 1100mV 85°C	-4.203	NIESPEŁNIONE
Hold	Slow 1100mV 85°C	0.241	SPEŁNIONE
Min Pulse Width	Slow 1100mV 85°C	3.857	SPEŁNIONE

Ujemny slack dla *setup* pozwala oszacować osiągalną częstotliwość maksymalną. Dla najgorszego narożnika (Slow 1100mV 85°C) raport wskazuje $F_{\max} \approx 70.41$ MHz, co odpowiada efektywnemu okresowi krytycznej ścieżki rzędu $T_{\text{crit}} \approx 14.203$ ns. Dla tego samego typu narożnika w niższej temperaturze (Slow 1100mV 0°C) uzyskano $F_{\max} \approx 71.15$ MHz.

5.6.4 Wnioski z kompilacji

Na podstawie tabel 5.9 oraz 5.10 można sformułować następujące wnioski:

- Projekt bez problemu mieści się w docelowym układzie Cyclone V (29% ALM, 7% pamięci bitowej, 4% DSP), co pozostawia zapas na ewentualną rozbudowę funkcjonalną.
- Ograniczeniem bieżącej implementacji jest czas krytycznej ścieżki: dla constraintu 100 MHz analiza *setup* wykazuje ujemny slack (-4.203 ns), a raportowany $F_{\max}$ wynosi ok. 70–71 MHz.

- Sprawdzenia *hold* oraz minimalnej szerokości impulsu są spełnione (slack dodatni), co wskazuje na brak problemów z nadmiernie krótkimi ścieżkami lub generacją zegara.

W kolejnych etapach pracy (lub jako kierunek rozwoju) możliwa jest optymalizacja architektury i/lub logiki, aby skrócić ścieżki krytyczne (np. przez dodatkowe potokowanie, reorganizację buforowania lub redukcję logiki kombinacyjnej na ścieżkach sterujących), jednak nie jest to wymagane do wykazania poprawnej synteżowalności projektu i oceny jego realizowalności w wybranym FPGA.

5.6.5 Szacowana przepustowość toru na podstawie $F_{\max}$ i latencji symulacyjnej

Wyniki analizy czasowej wskazują, że dla najgorszego narożnika (Slow 1100mV 85°C) osiągalna częstotliwość wynosi w przybliżeniu $F_{\max} \approx 70.41$ MHz. Równocześnie, w rozdziale poświęconym weryfikacji wyznaczono latencje w cyklach zegara dla wybranych przypadków testowych. Pozwala to oszacować realnie osiągalną przepustowość toru.

Metryka end-to-end (goodput TB). Dla pojedynczego przypadku testowego czas przetworzenia całego TB można przybliżyć jako:

$$T_{E2E} = \frac{N_{\text{cykli}}}{F_{\max}}, \quad N_{\text{cykli}} = t_{\text{out,last}} - t_{\text{in,first}} + 1,$$

gdzie $t_{\text{in,first}}$ oznacza cykl wystąpienia pierwszego ważnego słowa na wejściu toru, a $t_{\text{out,last}}$ cykl wystąpienia ostatniego ważnego słowa na wyjściu (z `out_last=1`). Efektywna przepustowość w sensie bitów informacji TB na sekundę wynosi:

$$R_{\text{TB}} = \frac{A}{T_{E2E}} = \frac{A \cdot F_{\max}}{N_{\text{cykli}}},$$

gdzie A to długość TB w bitach (z metadanych przypadku).

W tabeli 5.11 zestawiono oszacowaną przepustowość R_{TB} dla wybranych przypadków, przy założeniu $F_{\max} = 70.41$ MHz.

Uwagi interpretacyjne. Wartości w tabeli 5.11 opisują przepustowość przy aktualnym modelu pracy toru, w którym przetwarzanie obejmuje buforowanie i obliczenia wewnętrzne przed fazą emisji, a pełny wynik jest wypychany po zakończeniu przetwarzania wszystkich CB. Dla krótkich ramek (np. A57) narzut sterowania i faz obliczeń dominuje, co obniża R_{TB} . Dla dłuższych ramek goodput rośnie i stabilizuje się na poziomie kilkudziesięciu Mb/s przy ograniczeniu częstotliwością $F_{\max}$.

Tabela 5.11: Oszacowana przepustowość end-to-end toru PDSCHTx w sensie bitów TB (goodput), dla $F_{\max} = 70.41$ MHz.

CASE	A [bit]	N_{cykli}	T_{E2E} [μs]	R_{TB} [Mb/s]
A57	57	1916	27.21	2.09
A3187	3187	5323	75.60	42.16
A4219	4219	7029	99.83	42.26
A12107	12107	22223	315.62	38.36
A18371	18371	30203	428.96	42.83
A26841	26841	39517	561.24	47.82
A59761	59761	82681	1174.28	50.89

Rozdział 6

Podsumowanie i wnioski

- uzyskane wyniki w świetle postawionych celów i zdefiniowanych wyżej wymagań
- kierunki ewentualnych danych prac (rozbudowa funkcjonalna ...)
- problemy napotkane w trakcie pracy

Bibliografia

- [1] Evgeni Stavinov. „A Practical Parallel CRC Generation Method”. W: *Circuit Cellar* 234 (2010). January 2010 – Issue 234, s. 38–45. URL: <https://cc-webshop.com/products/circuit-cellar-issue-234-january-2010-pdf> (dostęp 15.10.2025).

Dodatki

Spis skrótów i symboli

DNA kwas deoksyrybonukleinowy (ang. *deoxyribonucleic acid*)

MVC model – widok – kontroler (ang. *model-view-controller*)

N liczebność zbioru danych

μ stopnień przyleżności do zbioru

$\mathbb{E}$ zbiór krawędzi grafu

$\mathcal{L}$ transformata Laplace’a

Źródła

Jeżeli w pracy konieczne jest umieszczenie długich fragmentów kodu źródłowego, należy je przenieść w to miejsce.

```
1 if (_nClusters < 1)
2     throw std::string ("unknown_number_of_clusters");
3 if (_nIterations < 1 and _epsilon < 0)
4     throw std::string ("You should set a maximal number of
        iteration or minimal difference — epsilon.");
5 if (_nIterations > 0 and _epsilon > 0)
6     throw std::string ("Both number of iterations and minimal
        epsilon set — you should set either number of iterations
        or minimal epsilon.");
```

Lista dodatkowych plików, uzupełniających tekst pracy

W systemie do pracy dołączono dodatkowe pliki zawierające:

- źródła programu,
- dane testowe,
- film pokazujący działanie opracowanego oprogramowania lub zaprojektowanego i wykonanego urządzenia,
- itp.

Spis rysunków

3.1	Interfejs wejściowy układu.	8
3.2	Interfejs wyjściowy układu.	9
4.1	Schemat blokowy modułu <code>crc_serial</code> liczącego CRC szeregowo.	16
4.2	Schemat blokowy modułu <code>crc_tb_parallel</code> liczącego CRC równolegle. . .	17
4.3	Schemat blokowy modułu <code>LDPC_prep</code> przygotowującego dane do kodowania LDPC.	22
4.4	Schemat blokowy modułu <code>LDPC_encode</code>	27
4.5	Integracja toru w module <code>PDSCHTx</code>	33

Spis tabel

5.1	Parametry wybranych przypadków testowych (metadane MATLAB). . . .	44
5.2	Metryki czasowe modułu <code>crc_serial</code> (cykle).	44
5.3	Metryki czasowe modułu <code>crc_tb_parallel</code> (cykle).	44
5.4	Metryki czasowe modułu <code>LDPC_prep</code> (cykle).	45
5.5	Metryki czasowe modułu <code>LDPC_encode</code> (cykle).	45
5.6	Metryki czasowe modułu top-level <code>PDSCHTx</code> (cykle).	46
5.7	Narzut wynikający z pracy na magistrali.	47
5.8	Konfiguracja kompilacji i docelowego układu FPGA.	47
5.9	Wykorzystanie zasobów po kompilacji (Fitter Summary).	48
5.10	Podsumowanie analizy czasowej (TimeQuest).	48
5.11	Oszacowana przepustowość end-to-end toru <code>PDSCHTx</code> w sensie bitów TB (goodput), dla $F_{\max} = 70.41$ MHz.	50